

BMM Developer Notes

Table of contents

Overview	5
1 System setup	8
1.1 Setting up a local fork of the bmm repository	8
1.2 Prerequisites	8
1.3 Package development with devtools	9
1.4 Untracked scratch work	10
2 Git and Github workflow	11
2.1 Git commands	11
2.2 Pull requests	12
2.3 AI-assisted contributions	12
2.3.1 What review will send back	12
3 BMM code structure	14
3.1 The main workhorse - <code>bmm()</code>	14
3.2 The <code>bmmformula</code> class	15
3.3 Models	15
3.3.1 Links	17
3.4 Model domains and class chains	19
3.5 S3 methods	20
3.6 Data flow: the attribute bridge pattern	21
3.7 Re-running <code>check_data()</code> in <code>update()</code>	22
3.8 Default priors structure	23
3.9 Suppressing <code>mu</code> for models without a location parameter	24
3.10 The <code>init_ranges</code> field and <code>create_initfun()</code>	24
3.11 Within-chain threading	25
3.12 Version tables	27
M3 nests the table one level deeper	27
Not every model uses either pattern	28
3.13 File organization	28
R/bmm.R	28
R/helpers-*.R	28
R/bmmformula.R	29
R/model_*.R	29

R/distributions.R	30
Post-processing	30
R/data_check.R, R/bmm_setup.R	30
R/utils.R, R/brms-misc.R, R/summary.R	30
inst/stan_chunks/	31
4 Example model file	34
4.1 The Interference Measurement Model (IMM)	34
4.1.1 Model definition	34
4.1.2 Model alias	36
4.1.3 check_data() methods	36
4.1.4 configure_model() methods	37
4.2 The Signal Discrimination Model (SDM)	38
4.2.1 Model definition	38
4.2.2 check_data() methods	38
4.2.3 configure_model() methods	39
4.2.4 Post-processing methods	40
4.3 The Diffusion Decision Model (DDM)	40
4.3.1 Model definition	40
4.3.2 User-facing alias	41
4.3.3 bmf2bf method	41
4.3.4 configure_model method	41
4.3.5 check_data method	41
4.4 The EZ-Diffusion Model (EZDM)	42
4.4.1 Version table	42
4.4.2 bmf2bf for aggregate data	42
4.4.3 Version-specific configure_model	43
5 Adding a new model	61
What use_model_template() does	61
What the template leaves for you	61
5.1 Example	62
5.2 Filling in the template	64
5.3 Before you open the pull request	66
5.4 Failure modes to expect	67
5.5 Testing	68
5.5.1 Parameter recovery	69
5.5.2 Distribution functions	69
5.6 Post-processing infrastructure	69
5.7 Add an example dataset	70
5.8 Add an article	70

6	Coding conventions	71
6.1	Errors, warnings and messages	71
6.2	Dependencies	72
6.3	Code shape	73
6.4	S3 conventions	74
6.5	Where to validate arguments	74
6.6	<code>log_lik</code> , <code>posterior_predict</code> and <code>posterior_epred</code>	75
6.7	Naming traps	76
6.8	Documentation and the checks that enforce it	77
6.9	Tests	78
6.9.1	Parameter recovery, and the circularity trap	78

Overview

Last update: 03.10.2026

This guide aims to help developers contribute new models to `bmm`. It is a work in progress and will be updated as the package evolves. It explains how to set-up your system for package development, the structure of the package, and the workflow for contributing new models to the package.

The `bmm` package supports models for four kinds of tasks: continuous reproduction (mixture models, IMM, SDM), categorical recall and n-AFC decisions (M3), detection, recognition and confidence judgments (the signal detection models), and choices with response times (DDM, EZDM, Censored-Shifted Wald). `supported_models()` lists them. The package requires **R** $\geq 4.1.0$ and uses the native pipe operator `|>` throughout. Further models are in development on feature branches; they are not part of a release and not covered here until they are.

This guide describes `bmm` at the **v1.4.0** tag. Anything merged after that tag may not be reflected here yet, and the version on CRAN can lag behind the tag while a submission is being processed. If you run into problems, don't hesitate to [open an issue on github](#).

We follow a [github flow workflow](#). The repository contains two main branches:

- Master (contains the latest released stable version of the `bmm` package)
- Develop (contains the latest stable development branch)

All new feature development should occur on an independent branch from Develop. If you want to contribute a new model to the `bmm` package, you need to fork the repository, create a new branch for your model off Develop, extensively test the model, and eventually submit a pull request into the Develop branch of the main repository.

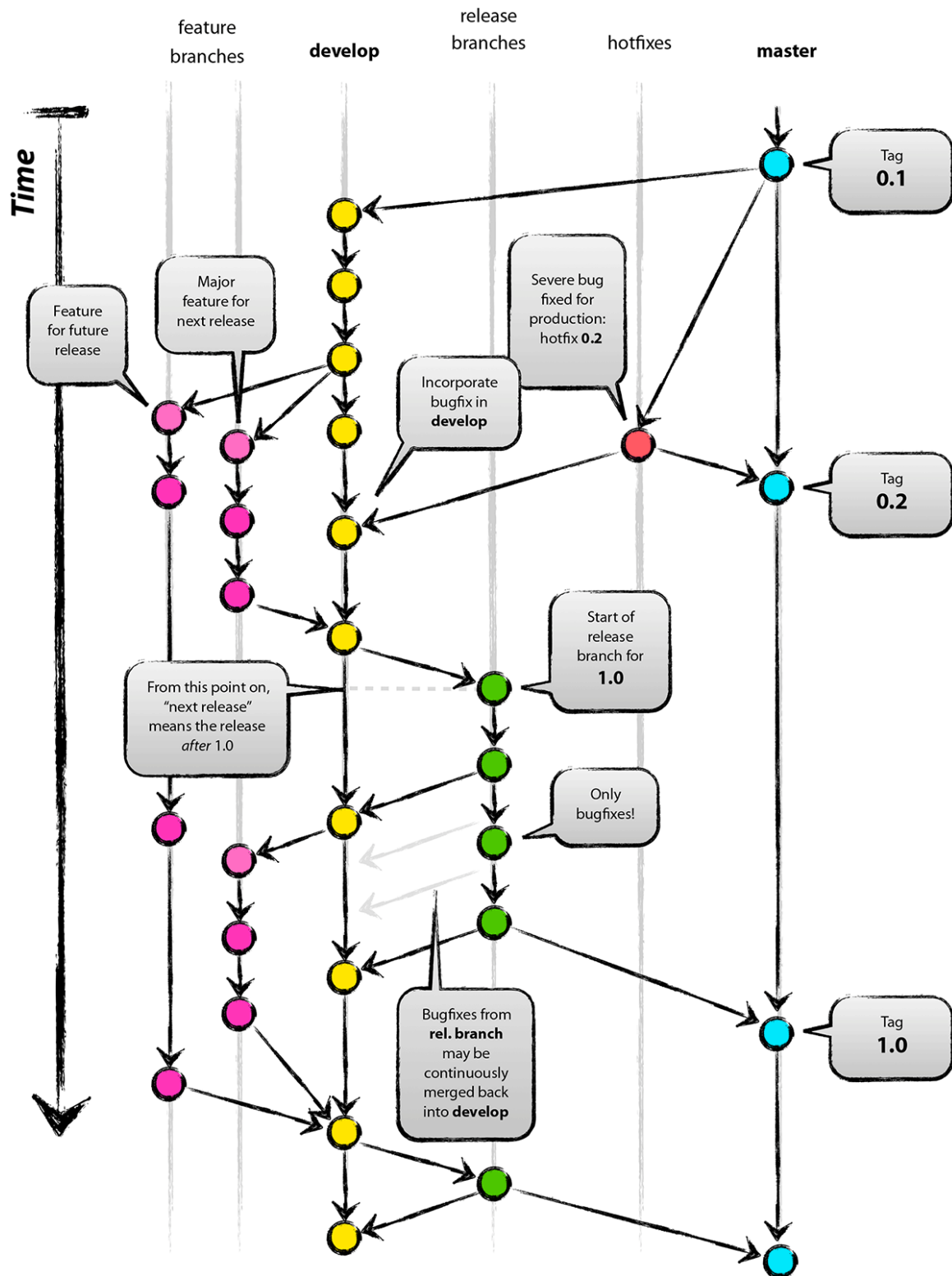
How that pull request is reviewed:

- One reviewer approval is required to merge. Two are required when the pull request changes shared infrastructure rather than a single model — the pipeline helpers under `R/helpers-*.R`, or the exported API — because those changes affect every model rather than only yours.

- A pull request that adds a new model is approved by a human who has installed your branch and fitted the model. Reading the diff is not sufficient for a new model, because the failure modes that matter (a likelihood that compiles but samples badly, priors that make the model unidentifiable, initial values that hit the boundary) only show up in a fit.

Accordingly, make the model easy to fit for whoever reviews it: include a short, runnable example with a small simulated dataset, and say in the pull request how long it takes to fit and what output to expect.

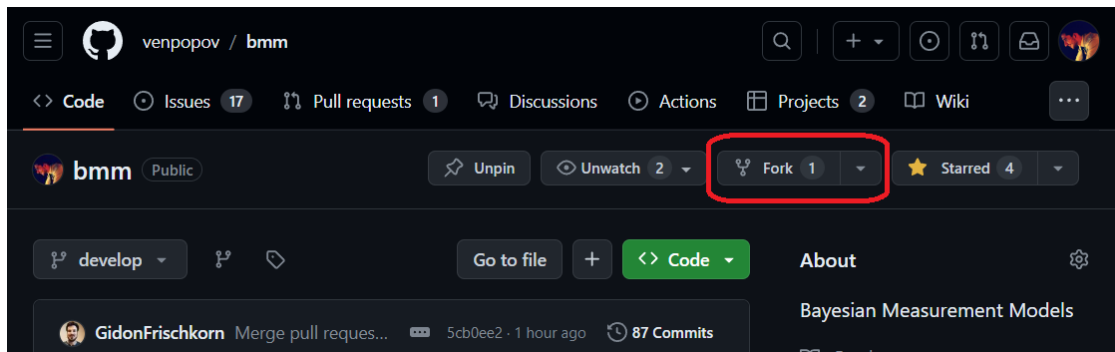
Once your changes are merged into the Develop branch, they will be included in the next release of the package.



1 System setup

1.1 Setting up a local fork of the bmm repository

1. Fork the bmm github repository. This will create a copy of the current development branch into your own github account



2. [Clone](#) your fork to your local machine
3. [Create a new branch](#) for your model off `develop`. Open an issue first and name the branch after it, as `<issue-number>-short-description` (for example `349-lba-model`). That is what the branches in the main repository look like, and it makes the branch, the issue and the eventual pull request easy to line up.

1.2 Prerequisites

The bmm package requires **R** $\geq 4.1.0$ (for native pipe `|>` support). The package works with both `rstan` and `cmdstanr` backends. `cmdstanr` is generally recommended for faster compilation and better error messages. You can install it with:

```
install.packages("cmdstanr", repos = c("https://stan-dev.r-universe.dev", getOption("repos")))
cmdstanr::install_cmdstan()
```

Once bmm is loaded, `bmm_setup()` checks the C++ toolchain, `cmdstanr` with `CmdStan`, `rstan`, and the backend `bmm()` will use; it then compiles and samples a small test model and suggests a fix for every check that failed. Run it before your first fit.

1.3 Package development with devtools

Development runs on the `devtools` package, and the steps below are the same whichever editor you work in. The repository ships a `bmm.Rproj` file, so opening it starts a session with the package root as the working directory in RStudio or Positron; opening the directory in any other editor, or starting R in it from a terminal, gets you to the same place. A great tutorial on package development can be found [here](#). Below is a summary of the most important steps

1. Make sure you have the `devtools` package and a few others installed and loaded

```
install.packages(c("devtools", "roxygen2", "testthat", "knitr"))
library(devtools)
install_dev_deps()
```

Optionally, to avoid loading `devtools` in every session, you can add the following to your `.Rprofile`

```
if (interactive()) {
  suppressMessages(require(devtools))
}
```

As noted [here](#), you can create and open an `.Rprofile` file, if you don't already have one with

```
use_devtools()
```

2. Load the current version of the `bmm` package based on your local files

```
load_all() # ctrl+shift+L in RStudio
```

you can use this command whenever you make changes to the package code to see the changes in action. You should not call `library(bmm)` or source the files manually, as this will load the installed version of the package, not the one you are developing.

3. Make any changes to the package code that you need to make (elaborated in the next section)
4. Use `check()` to check the package for errors and warnings

```
check()
```

you should always ensure that `check()` produces no errors before submitting a pull request

5. Use `document()` to update the documentation

```
document()
```

1.4 Untracked scratch work

Development produces files that belong on your machine rather than in the package: exploratory scripts, fitted model objects, scratch data. bmm's `.gitignore` excludes `local/` for this purpose — `AGENTS.md` describes it as machine-specific scratch that is neither synced nor shared — so anything you put there stays out of your commits. Nothing requires you to use it; any location git ignores will do.

Parameter recovery scripts are the main thing you will keep out of the package this way. What the study has to show, and how it reaches the reviewer given that it is not committed, is in [Chapter 6](#).

2 Git and Github workflow

(This is not meant to be a complete guide to git, but rather a short summary of key commands)

After your fork the `bmm` repository and create a branch for your new model/feature, you can follow a typical git workflow. As you make changes and add new files, you will want to:

2.1 Git commands

[Add](#) your changes to git, [commit](#) them and then [push](#) your changes to your forked repository. You can run these commands from any terminal, including the one built into your editor, with the project as the working directory

Add changed files to the staging area. It is best to add specific files rather than using `git add *` or `git add .`, which can accidentally stage unintended files (e.g., `.env`, large data files, or fitted model objects):

```
git add R/model_my_model.R inst/stan_chunks/my_model_funs.stan
```

Commit the changes to the local repository

```
git commit -m "A short message describing the changes you made"
```

Push the changes to your forked repository

```
git push
```

You can (and should) repeat this process as many times as you need to before submitting a pull request. This will allow you to make many small changes and test them before submitting a pull request. Ideally each commit should be a small, self-contained change that can be easily reviewed.

2.2 Pull requests

When you are ready you can open a pull request from your forked repository to the main bmm repository. You can do this from the github website. Make sure to select the Develop branch as the base branch and your feature branch as the compare branch. You should add a detailed description of your changes, including the motivation for the changes and any relevant context. You should also mention any issues that your pull request resolves.

2.3 AI-assisted contributions

AI assistance is welcome. The maintainers use it. This section is not a warning against it, but a statement of what we need from you so that a pull request written with help from a language model can be reviewed like any other.

You are the author of every line you submit. Concretely, that means three things. You can explain what each function does and why it is written the way it is. You have run it — a real fit on real or simulated data, not only the test suite. And you answer for it in review, including the parts you did not type yourself.

Say in the pull request body which parts were AI-assisted and how you verified them. This is not a disclaimer and it does not count against you. It tells the reviewer where to look hardest, which makes the review faster and better, and it is the difference between a reviewer trusting the rest of the diff and checking all of it twice.

What the review does with it. A human approves, and for a new model that human has installed your branch and fitted the model. The disclosure tells them where to read closely; it does not shift the work onto a tool.

2.3.1 What review will send back

AI-generated code in bmm fails in recognizable ways. These are the ones that come up most often, and they are all cheap to fix before you open the pull request rather than after:

- **Defensive validation sprinkled through internals.** A model generated with help tends to acquire argument checks in `.model_*`, in `configure_model.*` and in every private helper. Validation belongs in the exported constructor, because that is the only place where the argument named in the message is the argument the user passed. See [Chapter 6](#).
- **Base `stop()`, `warning()` and `sprintf()`.** These are what a model trained on general R code reaches for. bmm uses `stop2()`, `warning2()`, `stopif()`, `warnif()` and `glue::glue()`, and the reasons are not stylistic: `message2()` respects `options(bmm.silent)` and `stopif()` interpolates in the caller's frame.

- **Explicit `return()` on the last line**, and `%>%` where the package uses `|>`.
- **Comments that restate the code.** `# construct the family` above a `custom_family()` call is noise. A comment should record why a line is the way it is, especially when it looks removable and is not.
- **A parameter recovery script that simulates and fits with the same function.** This is the one that matters most, because it produces a result that looks like evidence. If you generate data with your own `r*()` function and fit a likelihood built from your own `d*()` function, any error common to both cancels and recovery looks clean whether the model is right or wrong. Use an independent generator.

The last one is worth the extra sentence: a recovery study that cannot fail tells the reviewer nothing, and it is the artifact they will ask for first.

3 BMM code structure

Adding a new model is straightforward using the `use_model_template()` function, which will be described in the next section. You do not have to edit any of the files below, but it will be helpful to understand the structure of the package.

3.1 The main workhorse - `bmm()`

The main function for fitting models is `bmm()`. This function is the main entry point for users to fit models. It is set-up to be independent of the specific models that are implemented in the package.

It calls several subroutines, implemented as generic S3 methods, to:

- `configure_options()` - configure local options for fitting, such as parallel sampling
- `check_model()` - check if the model exists and is valid, including the user's `links`
- `check_data()` - check whether the data contains all necessary information and transform it
- `check_formula()` - check if the formula is specified correctly
- `configure_model(model, data, formula)` - configure the model specification (formula, family, Stan code) to pass to brms
- `configure_prior()` - set the default priors for the model and combine them with the user prior
- `create_initfun()` - create an initial values function for MCMC sampling. It receives the final prior, because a parameter the prior holds constant does not exist in the Stan code and must get no initial value
- `postprocess_brm()` - post-process the fitted model (e.g., adjust link functions, add metadata)

`brms_frame_args(dots)` picks out the `brm()` arguments that change the model frame (`data2`, `knots`, `drop_unused_levels`) and hands them to the two steps that ask brms about the model before the fit.

The fitted model is cached when the user passes `file`. With `file_refit = "on_change"`, the cached fit is reused only if the Stan code and data are unchanged, which is why that comparison sits after configuration, just before `brm()`. One consequence for you: `configure_model()` must generate the same Stan code for the same inputs.

The first line worth a second look is `local_brms_threads(dots)`. `threads` is a `brms::brm()` argument and reaches it through ... like any other, so `bmm()` does not need to know about it — except that a model may need to *generate different Stan code* when threading is on. `local_brms_threads()` normalizes whatever the user passed (a bare number becomes `brms::threading(n)`, `threads = NULL` turns threading off) and sets the `brms.threads` option for the duration of the call, so that a `configure_model()` method further down the pipeline can read it. See Section 3.11.

3.2 The `bmmformula` class

Before understanding models, it's important to understand how users specify what to estimate. The `bmmformula` (alias: `bmf()`) is a list of formulas, one per model parameter. Unlike `brmsformula`, there is no response variable on the left-hand side — the response is specified when constructing the model object.

```
# Each formula predicts one model parameter
f <- bmmformula(
  kappa ~ 0 + set_size + (1 | id),
  c ~ 1 + (1 | id),
  a ~ 1
)

# Equivalent shorthand
f <- bmf(kappa ~ 0 + set_size + (1 | id), c ~ 1 + (1 | id), a ~ 1)
```

Parameters can be **fixed to a constant** using `=` instead of `~`:

```
f <- bmf(kappa ~ condition, a = 0.5) # 'a' is fixed to 0.5, not estimated
```

Parameters **not mentioned** in the formula are automatically given an intercept-only formula (`param ~ 1`) and a message is printed.

The `bmmformula` is converted to a `brmsformula` during model configuration via the `bmf2bf()` S3 method. This conversion adds the response variable as the first formula line and translates each parameter formula into either `brms::lf()` (linear) or `brms::nlf()` (non-linear) components.

3.3 Models

All models in the package are defined as S3 classes and follow a strict template. This allows us to implement general methods for handling model fitting, data checking, and post-processing. Each

model has an internal constructor (`.model_<name>()`) that defines the model and its parameters, and a user-facing alias. To keep the specification easy to read and edit, the parameters, links, fixed parameters, priors, and initialization ranges live in a separate `.<name>_defaults` list — or, for models with several versions, in a `.<name>_version_table` (see [Version tables](#)) — and the constructor spells out each field of the model object inline, reading from that list. For a complete example model file and an explanation, see [Section 4](#). The general model template looks like this:

```
# the model's parameter specification (one such list per version in a version table)
.my_new_model_defaults <- list(
  parameters = list(
    par1 = "Parameter 1 = description of parameter 1",
    par2 = "Parameter 2 = description of parameter 2"
  ),
  links = list(par1 = "identity", par2 = "log"),
  fixed_parameters = list(mu = 0),
  priors = list(
    par1 = list(main = "normal(0, 1)", effects = "normal(0, 0.5)", sd = "exponential(1)"),
    par2 = list(main = "normal(0, 0.5)", effects = "normal(0, 0.5)", sd = "exponential(2)")
  ),
  init_ranges = list(par1 = c(-1, 1), par2 = c(0.5, 1.5))
)

.model_my_new_model <- function(resp_var1 = NULL, required_arg1 = NULL,
                                required_arg2 = NULL, links = NULL,
                                call = NULL, ...) {
  out <- structure(
    list(
      resp_vars = nlist(resp_var1),
      other_vars = nlist(required_arg1, required_arg2),
      domain = "",
      task = "",
      name = "",
      citation = "",
      version = "NA",
      requirements = "",
      parameters = .my_new_model_defaults[["parameters"]],
      links = .my_new_model_defaults[["links"]],
      fixed_parameters = .my_new_model_defaults[["fixed_parameters"]],
      default_priors = .my_new_model_defaults[["priors"]],
      init_ranges = .my_new_model_defaults[["init_ranges"]]
    ),
    class = c("bmmodel", "my_new_model"),
```

```

    call = call
  )
  out <- set_links(out, links)
  out
}

```

The `.my_new_model_defaults` list holds the parameter specification (one such list per version in a version table). Its fields are:

- **parameters** - Named list of model parameters with descriptions. List only parameters a user can estimate or fix; a dummy that exists only for brms (see Section 3.9) stays out
- **links** - Named list of link functions for each parameter (see Section 3.3.1)
- **fixed_parameters** - Named list of parameters fixed to constant values (e.g., `list(mu = 0)`)
- **priors** - Named list with **main** (intercept), **effects** (slopes) and **sd** (random-effect SDs) priors per parameter (see Section 3.8). The constructor stores this under the model object's **default_priors** field
- **init_ranges** - Named list of `c(lower, upper)` vectors for MCMC initialization (see Section 3.10)

The constructor copies these into the model object (renaming **priors** to **default_priors**) alongside the response and auxiliary variables (**resp_vars**, **other_vars**), the model metadata (**domain**, **task**, **name**, **citation**, **requirements**), and the **version**. A test checks that every constructor lists these fields in exactly this order and can be called with no arguments.

domain does more than document: **supported_models()**, the README and the website group models into tasks by it, so reuse the string of the models yours belongs with. Keeping the fields inline means a new field on the model object is a one-line addition you can see and copy directly; the trade-off is that older models need updating when a field is added (a periodic, mechanical refactor).

3.3.1 Links

Each parameter's link maps the scale Stan samples on to the scale the parameter is interpreted on. bmm implements "identity", "log", "softplus", "log1p", "logm1", "inverse", "sqrt", "logit", "probit", "cloglog" and "tan_half"; a model may also declare "loglog" or "softmax" as its own default, but users cannot ask for them. "softplus" is an opt-in alternative to "log" for positive parameters: it grows linearly rather than exponentially, so predictor effects are additive on the natural scale and large values do not overflow.

Users change links through the alias's **links** argument, and the constructor assigns them with **set_links()**. It refuses a name that is not a parameter (a one-edit typo is repaired, with a warning), refuses a link the model cannot apply, and warns when the new link admits values

the default excludes, such as `identity` for a positive parameter. `check_model()` runs the same validation on links assigned after construction (`model$links <- ...`).

Which links a user may change is up to the model. The default is all of them. A model that writes its links into the family, the Stan code or the non-linear formulas, rather than reading them from `model$links` at fit time, declares that with a `settable_links()` method returning `character(0)` or the subset that does work; otherwise a user's link would show in `print()` and in the priors while the sampler ignored it. SDM, the mixture models and the IMM return `character(0)`; the SDT models return the parameters whose links reach the likelihood.

Each model is accompanied by a user-facing alias, the documentation of which is generated automatically based on the info list in the model definition.

```
# user facing alias
# information in the title and details sections will be filled in
# automatically based on the information in the .model_modelname()$info
#' @title `r .model_my_new_model()$name`
#' @name my_new_model
#' @details `r model_docs(.model_my_new_model())`
#' @param resp_var1 A description of the response variable
#' @param required_arg1 A description of the required argument
#' @param required_arg2 A description of the required argument
#' @param ... used internally for testing, ignore it
#' @return An object of class `bmmmodel`
#' @keywords bmmmodel
#' @export
#' @examples
#' \dontrun{
#' # put a full example here (see 'R/model_ddm.R' for an example)
#' }
my_new_model <- function(resp_var1, required_arg1, required_arg2,
                          links = NULL, ...) {
  call <- match.call()
  stop_missing_args()
  .model_my_new_model(resp_var1 = resp_var1, required_arg1 = required_arg1,
                      required_arg2 = required_arg2, links = links,
                      call = call, ...)
}
```

For a versioned model, the alias instead carries a `version = c("v1", "v2")` argument validated with `version <- match.arg(version)`, and passes the chosen `version` on to the constructor (see `cswald` for a worked example). Every versioned model validates this way, so an unknown version fails where the user typed it and an unambiguous abbreviation works. `use_model_template()` generates whichever variant you ask for via its `versions` argument.

Then users can fit the model using the `bmm()` function, and the model will be automatically recognized and handled by the package:

```
fit <- bmm(formula = my_bmmformula,
           data = my_data,
           model = my_new_model(resp_var1, required_arg1, required_arg2))
```

3.4 Model domains and class chains

Models use hierarchical S3 class chains that encode the model taxonomy. The class chain determines which S3 methods are dispatched. The general pattern is:

```
class = c("bmmmodel", <domain/intermediate>, <model>, <version>)
```

The current model families and their class chains are:

Circular / Visual working memory models:

- mixture2p: `c("bmmmodel", "circular", "mixture2p")`
- mixture3p: `c("bmmmodel", "circular", "non_targets", "mixture3p")`
- imm (full): `c("bmmmodel", "circular", "non_targets", "imm", "imm_full")`
- sdm: `c("bmmmodel", "circular", "sdm", "sdm_simple")`

Response time / Decision models:

- ddm: `c("bmmmodel", "ddm")`
- ezdm (3par): `c("bmmmodel", "ezdm", "ezdm_3par")`
- cswald (simple): `c("bmmmodel", "cswald", "cswald_simple")`

Categorical response models:

- m3 (ss): `c("bmmmodel", "m3", "m3_ss")`

Signal detection models:

- sdt_yn: `c("bmmmodel", "sdt", "sdt_yn");` likewise `sdt_mafc`, `sdt_ranking` and `sdt_cdp`
- sdt_rating (standard): `c("bmmmodel", "sdt", "sdt_rating", "sdt_rating_standard")`

Intermediate classes like "circular", "non_targets" and "sdt" allow shared methods (e.g., `check_data.circular` validates that responses are in radians, `check_data.non_targets` validates non-target feature columns, `check_formula.sdt` refuses the stimulus column as a predictor, and the shared SDT helpers live in `R/helpers-sdt.R`). Some infrastructure also keys on them: a model with class "circular" refuses `posterior_epred()`. S3 methods chain via `NextMethod()` from general to specific: dispatch walks the class vector left to right, so `check_data.bmmmodel` runs first and the model-specific method runs last. See the worked dispatch chain in Chapter 4.

3.5 S3 methods

The package uses S3 methods to handle different models. This means that the same function can behave differently depending on the class of the object it is called with. For example, the `configure_model(model, data, formula)` function called by `bmm()`, is generally defined as:

and it will call a function `configure_model.modelname()` that is specified for each model. The same is true for other functions, such as `check_data()`, `check_formula()`, `configure_prior()`, `create_initfun()`, and `postprocess_brm()`. This allows us to add new models without having to edit the main fitting function, `bmm()`.

The key S3 generics in the pipeline are:

Generic	Signature	Defined in
<code>check_model</code>	<code>(model, data = NULL, formula = NULL)</code>	<code>helpers-model.R</code>
<code>check_data</code>	<code>(model, data, formula)</code>	<code>helpers-data.R</code>
<code>check_formula</code>	<code>(model, data, formula)</code>	<code>bmmformula.R</code>
<code>configure_model</code>	<code>(model, data, formula)</code>	<code>helpers-model.R</code>
<code>configure_prior</code>	<code>(model, data, formula, user_prior, ...)</code>	<code>helpers-prior.R</code>
<code>create_initfun</code>	<code>(model, data, formula, prior = NULL, ...)</code>	<code>helpers-inits.R</code>
<code>postprocess_brm</code>	<code>(model, fit, ...)</code>	<code>helpers-postprocess.R</code>
<code>bmf2bf</code>	<code>(model, formula)</code>	<code>bmmformula.R</code>

Match these signatures exactly in your methods, including the `...` where the generic has one and the `= NULL` defaults on `check_model`. R CMD check compares every method against its generic, and a method that drops an argument the generic declares is reported under “checking S3 generic/method consistency”.

Beyond the pipeline, a handful of generics let a model extend what happens after the fit: `revert_check_data()` (see Section 3.7), `settable_links()` (see Section 3.3.1), `pp_observables()` and `pp_simulate()`, `native_transform()` and `data_check_findings()`. Each has a default that covers most models; Chapter 5 says when to write a method.

The `bmf2bf` generic converts a `bmmformula` to a `brmsformula`. Models with a single response use the default `bmf2bf.bmmmodel` method. Models with response wrapping (e.g., `bmf2bf.ddm` builds `rt | dec(response) ~ 1`) or several response variables (e.g., `bmf2bf.ezdm_3par` and `bmf2bf.sdt_yn` pack them with `vreal()`, `vint()`, `trials()`) define their own.

The `configure_model` method must return a named list via `nlist()`. The required and optional fields are:

- **formula** (required) — the `brmsformula` object, with `formula$family` set to the appropriate family
- **data** (required) — the data frame (possibly with additional columns added during configuration)
- **stanvars** (optional) — custom Stan code, needed only for models with `brms::custom_family()`. Models using built-in brms families (e.g., `brms::mixture()` for the IMM) do not need `stanvars`.

Initial values are **not** returned by `configure_model` — they are handled by `create_initfun()` in a separate pipeline step.

3.6 Data flow: the attribute bridge pattern

A key architectural pattern is how information flows between `check_data()` and `configure_model()`. Since these are separate S3 methods, they cannot share local variables. Instead, `check_data` stores computed values as **attributes on the data frame**, and `configure_model` retrieves them.

```
# In check_data.non_targets (R/helpers-data.R):
attr(data, "max_set_size") <- max_set_size
attr(data, "lure_idx_vars") <- lure_idx_vars

# Later, in configure_model.imm_full (R/model_imm.R):
max_set_size <- attr(data, "max_set_size")
lure_idx <- attr(data, "lure_idx_vars")
```

This is the way to pass model-specific metadata between pipeline steps. If your `check_data` method computes something that `configure_model` needs (e.g., the maximum set size, index variables, or transformed columns), store it as a data attribute.

Two qualifications. First, the infrastructure uses the same channel for its own bookkeeping: `check_data.bmmmodel` sets `data_name` and `checked` on the data frame, so do not assume the attributes you find there are all yours. Second, an attribute is not a guarantee. A `configure_model` method can be called directly, bypassing `check_data` entirely, and then the attribute is absent. `configure_model.sdm` handles this by recomputing rather than failing:

```
run_metadata <- attr(data, "sdm_run_metadata")
if (is.null(run_metadata)) {
  # Guard direct configure_model.sdm() calls that bypass check_data.sdm().
  run_metadata <- sdm_run_metadata(data, formula, model)
}
```

Whether to guard this way is a judgement call. It costs a little duplicated computation and it means the expensive part has to live in a function both methods can call, which is good structure anyway. If you do not guard, make the failure loud rather than silent.

3.7 Re-running `check_data()` in `update()`

`update(fit)` without `newdata` refits on the data stored in the fit. That is brms's model frame, `fit$data`: it holds only the variables the formula uses, and it is already the *output* of your `check_data()`. Running `check_data()` on it again fails for any model whose check adds, renames or packs columns — a count model that packs its responses into one matrix column, say.

So `update()` first calls `revert_check_data(model, data)` to bring the stored frame back into the shape the user's data had, then runs the usual `check_data()`. The default method returns the data unchanged, which is right for a `check_data()` that only validates. If yours changes columns, write a method in `R/update.R`. Two rules hold for every method:

- End with `NextMethod("revert_check_data")`, as `check_data` methods do, so that a method on a shared class (`non_targets`, `sdt`) still runs.
- brms keeps only the columns the fitted formula uses, so a column your `check_data()` needs may be missing from the stored frame (the `set_size` column of a non-target model, the `m` column of `sdt_mafc`). A method that has to recompute such a column names it in the `"rebuilt"` attribute of the data, and `update()` drops it again after the check so a new formula cannot pick it up. Append to that attribute; do not overwrite it, or you erase what a method earlier in the chain recorded.

`revert_check_data.m3` and the SDT methods are the examples. A test ("every supported model has a stored-frame case" in `tests/testthat/test-update.R`) requires each model to have a case in `stored_frame_cases()`; further tests in the same file run every case and check that the reverted frame reproduces the fit's Stan data.

3.8 Default priors structure

The `default_priors` field in model definitions has a specific structure that the `configure_prior()` method interprets:

```
default_priors = list(
  kappa = list(
    main = "student_t(5, 1.75, 0.75)", # prior for Intercept
    effects = "normal(0, 1)",          # prior for regression coefficients
    sd = "exponential(1)"              # prior for random effect SDs
  ),
  c = list(
    main = "student_t(5, 2, 0.75)",
    effects = "normal(0, 1)",
    sd = "exponential(1)"
  )
)
```

- **main:** Applied to the Intercept of the parameter. If the user suppresses the intercept (e.g., `kappa ~ 0 + condition`), this prior is applied to each level of the factor instead. Keep it the first element: without an intercept, it is found by position.
- **effects:** Applied to regression coefficients (slopes). If omitted, brms default priors are used for effects.
- **sd:** Applied to all random-effect standard deviations of this parameter. Every model in bmm declares one, except the custom version of M3, which has no default priors at all. If you omit it, the parameter falls back to brms' `student_t(3, 0, 2.5)`, which on a log link lets individual values vary by a factor of 12 around the group value. The shipped models use `exponential()` on the link scale: rate 1 for sensitivity, memory-strength, mixing-weight and identity-linked drift parameters; rate 2 for criteria and thresholds, boundary, non-decision time, start point and log-linked drift parameters; rate 4 for circular bias.

The priors use the same string format as `brms::set_prior()`. These defaults are automatically applied by `bmm()` unless the user explicitly overrides them via the `prior` argument. Whenever a model estimates correlations among random effects, they get `lkj(2)`; nothing in the model definition is needed for that.

For parameters listed in `fixed_parameters` that also appear in `parameters` (e.g., `mu1` in the IMM, which is fixed by default but can be estimated), the fixed value takes precedence unless the user provides a formula for that parameter.

3.9 Suppressing `mu` for models without a location parameter

The `brms` package requires every custom family to have a `mu` parameter (the location parameter). However, some models – particularly response time and decision models like DDM, EZDM, and `cswald` – do not have a natural location/`mu` parameter. These models suppress `mu` rather than arbitrarily promoting one of their parameters to it:

1. A dummy `mu` parameter is included in the family’s `dpars` with an `identity` link
2. The `mu` intercept is fixed to 0 by adding `mu = 0` to `fixed_parameters`
3. Users do not specify a formula for `mu` in their `bmmformula`
4. The model’s `bmf2bf` method constructs the first formula line differently (e.g., `rt | dec(response) ~ 1` for DDM, rather than using `mu` as a response)

This is purely a technical workaround to satisfy `brms` requirements; the fixed `mu = 0` ensures it has no effect on estimation.

Note

Earlier versions tracked this with a `void_mu` flag in the model definition. The flag was never read by the infrastructure – suppression is handled entirely by `fixed_parameters` and the family’s `dpars` – and it has been removed. If you see `void_mu` in older code or notes, it is dead and can be deleted.

3.10 The `init_ranges` field and `create_initfun()`

`brms`’ default initialization draws every parameter from a wide symmetric range on the sampling scale. When a model’s likelihood has hard boundaries or regions of zero density, that often places chains in impossible territory, causing immediate divergences or chain failures.

This is particularly relevant for response time models, where parameters have hard constraints relative to the observed data. For example, the non-decision time parameter `ndt` must be positive *and* smaller than the fastest observed RT – otherwise the implied decision time would be negative and the likelihood undefined. Similarly, boundary separation (`bound`) must be strictly positive and within a plausible range, and extreme drift rate values can cause numerical overflow in the Wiener likelihood.

The `init_ranges` field in the model definition addresses this by specifying narrow, safe windows on the native (pre-link-transform) scale:

```
init_ranges = list(
  mu      = c(-0.1, 0.1),
  drift   = c(-1, 1),
```

```
bound = c(1, 2),
ndt    = c(0.01, 0.05)
)
```

Each entry maps a parameter name to a `c(lower, upper)` range. `create_initfun.default()` (in `R/helpers-inits.R`) parses the Stan code for every parameter's dimensions and returns an initialization function that gives every Stan parameter it can size a value (the rest are left to the sampler):

1. Random effects start tight: standard deviations between 0.05 and 0.1, standardized effects near zero, correlation matrices at the identity.
2. Population-level parameters with an `init_ranges` entry start inside that range. The range is on the native scale (for softmax mixing weights, on the sampling scale), and it is transformed through the parameter's link and clipped to what the link can represent, so a range stays valid when the user changes the link. With predictors, the coefficients are solved so that each design cell starts inside the range.
3. Everything else gets brms' default draw.

So `init_ranges` is optional, and per parameter: declare a range for the parameters whose likelihood needs one, and leave out the rest. The mixture models and the IMM set theirs to the central 50% of the main default prior, which is a sensible default when you have no data-driven constraint.

The conversion between native and sampling scales is handled by `link_transform()` (defined in `R/helpers-parameters.R`). It supports every elementwise link bmm uses (softmax weights are transformed jointly elsewhere) and can apply transformations in either direction – from the scale on which parameters are interpretable (native) to the scale on which Stan samples (sampling), and back. Model developers rarely need to call it directly; it is used automatically by `create_initfun()` when generating initial values, and by the post-processing infrastructure when converting estimates back to the native scale.

3.11 Within-chain threading

Passing `threads` to `bmm()` asks brms to split each chain's likelihood evaluation across cores. brms slices the response and calls the family's likelihood inside `partial_log_lik`, where the row bounds `start` and `end` exist. Two things it does *not* slice, and a model has to handle itself:

- **A custom family's vars.** brms pastes them in as written, so a vectorized (`loop = FALSE`) likelihood would receive the whole vector next to a slice of the response. cswald therefore passes `vars = "dec[start:end]"` when threading and `"dec"` otherwise.

- **Stan code injected through `stanvar()`**, which has to be written differently depending on whether it ends up inside `partial_log_lik` or in the model block.

Both decisions rest on one predicate, `brms_slices_likelihood()` in `R/utils.R`. It reads the `brms.threads` option that `local_brms_threads()` set at the top of `bmm()`, normalizes it as `brms` does (a bare number means `threading(n)`), and is `FALSE` for `threading(force = TRUE)`: in that case `brms` compiles with threading support but emits *unsliced* code, so `start` and `end` do not exist. Call it in `configure_model()`; do not read the option yourself. `SDM`, `cswald` and `sdt_mafc` are the models that use it.

`SDM` is the fullest example, worth reading if your model has a likelihood with a normalizing constant computed over groups of rows:

1. Two versions of the chunk. `inst/stan_chunks/` holds both `sdm_simple_likelihood.stan` and `sdm_simple_likelihood_threaded.stan`. The threaded one calls a `_slice` variant of the same function and passes `start` and `end`, the row bounds `brms` makes available inside `partial_log_lik`:

```
// serial
target += sdm_simple_run_ldenom(c, kappa, COSN, G_sdm_runs,
                                sdm_run_start, sdm_run_count);

// threaded
target += sdm_simple_run_ldenom_slice(c, kappa, COSN, start, end,
                                       G_sdm_runs, sdm_run_start,
                                       sdm_run_count);
```

2. The predicate decides which one to read, at configuration time:

```
likelihood_file <- if (brms_slices_likelihood()) {
  "sdm_simple_likelihood_threaded.stan"
} else {
  "sdm_simple_likelihood.stan"
}
```

3. Data that the threaded function needs must also be declared for the partial-log-likelihood signature, via the `pll_args` argument to `stanvar()`. `SDM` passes "data matrix `COSN`" for its precomputed cosine matrix and, through the helper `sdm_stanvar_int_array()`, "data array[] int <name>" for the run-index arrays. A variable that is not declared in `pll_args` is simply not in scope inside `partial_log_lik`, and the model will not compile.

Test the generated Stan code with no threads, with `threads = 2`, and with `threading(2, force = TRUE)`; the last is the case you will not think to try. `test-model_sdm-stancode.R` does all three.

3.12 Version tables

Some models support multiple versions with different parameter sets. Two patterns are used:

If/else pattern (older, used by IMM): Version-specific parameters are selected via conditional logic within the model constructor.

Version table pattern (newer, used by EZDM, cswald and M3): Version-specific parameters, links, priors, and `init_ranges` are stored in a lookup table:

The model constructor then indexes into this table for each field, selecting the entry for the requested `version`:

```
parameters = .ezdm_version_table[[version]][["parameters"]],
links = .ezdm_version_table[[version]][["links"]],
...
```

The table is the single source of truth for the version-specific specification. This pattern is cleaner and easier to extend than if/else logic, and new models with multiple versions should prefer it — `use_model_template("mymodel", versions = c("v1", "v2"))` generates it for you.

M3 nests the table one level deeper

The shape above is not universal, and M3 is the exception you will hit if you build a model whose parameterization depends on something other than the version. In `.m3_version_table`, `links` and `priors` are indexed by the **choice rule** before the parameter:

```
links = list(
  simple = list(c = "log", a = "log"),
  softmax = list(c = "identity", a = "identity")
),
priors = list(
  simple = list(a = list(...), c = list(...)),
  softmax = list(a = list(...), c = list(...))
)
```

so the constructor indexes twice: `.m3_version_table[[version]][["links"]][[choice_rule]]`. Three further departures in the same constructor are worth knowing before you copy it as a template:

- `fixed_parameters` is not in the table at all. It is computed inline from the choice rule: `list(b = if (choice_rule == "softmax") 0 else 0.1)`.

- `parameters` is the table entry with a background-activation parameter `b` prepended in the constructor, because every M3 version has it.
- The table holds no `init_ranges`, and it has entries for `ss` and `cs` only. M3's default version is "custom", which is not in the table — the indexing returns NULL and the user supplies the specification. That is deliberate, not an omission.

If your model needs a second axis of variation, this is what it looks like. If it does not, keep the single-level shape the template generates.

Not every model uses either pattern

`sdm`, `mixture2p`, `mixture3p`, `imm` and most of the signal detection models carry no `.{model}_defaults` object and no version table. They write `parameters`, `links`, `fixed_parameters`, `default_priors` and `init_ranges` as literal lists inside the constructor. The separated specification is the one `use_model_template()` generates and the one to use for a new model; the inline models predate it or were written alongside it. Do not conclude the book is wrong when you open `R/model_sdm.R` and find no defaults list.

3.13 File organization

The `bmm` package is organized into several files. The main files are:

`R/bmm.R`

It contains the main function for fitting models, `bmm()`. This function is the main entry point for users to fit models. It is set-up to be independent of the specific models that are implemented in the package.

To add new models, you do not have to edit this file. The functions above are generic S3 methods, and they will automatically recognize new models if you add appropriate methods for them (see Chapter 5).

`R/helpers-*.R`

`R/helpers-data.R`, `R/helpers-inits.R`, `R/helpers-model.R`, `R/helpers-parameters.R`, `R/helpers-postprocess.R`, and `R/helpers-prior.R`

These files define the main generic S3 methods for checking data, creating initial value functions, configuring the model, extracting parameters, post-processing the fitted model, and combining priors. They contain the default methods for these functions, which are

called by `bmm()` if no specific method is defined for a model. A new model adds its own methods in its model file. The exceptions are a few central tables in `R/helpers-model.R` (`data_column_roles`, `response_annotations()`) and, where your `check_data()` changes columns, a `revert_check_data()` method in `R/update.R`; the checklist in Chapter 5 lists them.

`R/helpers-data.R` also contains several user-facing utility functions for response time data preparation. These follow a deliberate design philosophy: each function performs one well-defined task without combining too many features. They are composable – users can pipe them together into a workflow but also inspect intermediate results at each step:

- `flag_contaminant_rts()` – identifies likely contaminant trials by fitting a mixture of the RT distribution and a uniform contaminant process, returning per-trial posterior probabilities of contamination
- `validate_fast_guesses()` – takes flagged contaminants and tests whether fast responses show chance-level accuracy consistent with guessing, using a Bayesian test
- `ezdm_summary_stats()` – computes the summary statistics (mean RT, variance, accuracy) that the EZDM model requires as input, with options for robust estimation and contaminant handling

`R/bmmformula.R`

This file contains the definition of the `bmmformula` class, which is used to represent the formula for the model. It contains the `bmmformula()` function and its alias `bmf()`, which is used to create a new formula object.

In addition, it contains the definition of the `bmf2bf` S3 method that is used to convert a `bmmformula` object into a `brmsformula` object. This is necessary, as `brmsformula` objects are required to include the response variable in the first formula line. In contrast `bmmformula` objects only contain formulas predicting the parameters of a `bmmmodel`. The `bmf2bf` S3 method is used to perform this conversion and add the first formula line including the response variable in the `brmsformula` created during model configuration. Some models override this with custom methods (e.g., `bmf2bf.ddm`, `bmf2bf.ezdm_3par`).

`R/model_*.R`

Each model and its methods live in one file named `model_<model_name>.R`: the specification, the internal constructor, the user-facing alias, the model-specific S3 methods, and the `log_lik`, `posterior_predict` and `posterior_epred` functions of a custom family. `use_model_template()` creates the file for you (see Chapter 5). `supported_models()` lists the current models; `R/model_ddm.R` (flat specification), `R/model_cswald.R` (version table) and `R/model_sdt_yn.R` (aggregated counts) are good files to read first.

Code shared by a family of models goes in a helpers file named for it, as `R/helpers-sdt.R` does for the signal detection models.

R/distributions.R

The density (`d*`), random generation (`r*`) and, where available, distribution (`p*`) and quantile (`q*`) functions of each model. They are not used by the Stan fit, but they simulate data, back `log_lik`, `posterior_predict` and `posterior_epred`, and drive the plots. A custom family needs at least `d*` and `r*` here.

Post-processing

These files work on fitted `bmmfit` objects and need no edits for a new model; the extension points they offer are listed in [Chapter 5](#).

- `conditional_effects.R`, `emmeans.R` - marginal effects and estimated marginal means
- `helpers-postprocess.R` - `postprocess_brm()`, `posterior_epred()` and `fitted()`
- `helpers-parameters.R` - `parameter_info()`, `native_parameters()`, `native_transform()` and `link_transform()`
- `helpers-prior.R` - default priors, `default_prior()` and `prior_info()`
- `pp_check.R`, `pp_observables.R` - posterior predictive checks, including checks of several observables per model
- `helpers-analysis.R`, `plot.R` - the signal detection post-processing (ROC, AUC, latent distributions) and its plots
- `update.R`, `restructure.R` - refitting, and bringing fits saved by older versions up to date

R/data_check.R, R/bmm_setup.R

`bmm_data_check()` runs the validation pipeline on a dataset without compiling the model and prints a report; models add their own diagnostics through `data_check_findings()`. `bmm_setup()` checks the user's toolchain and Stan backend.

R/utils.R, R/brms-misc.R, R/summary.R

Various utility functions. Note that `R/brms-misc.R` is a read-only vendored copy from `brms` and should not be edited.

`inst/stan_chunks/`

The Stan code passed to `brms::stanvar()`: custom likelihoods and helper functions for models with a custom brms family. Files are named `{model_name}_{block}.stan` (e.g., `my_model_functions.stan`), and code shared by several models gets a file of its own (`sdt_dist_funs.stan`).

When writing Stan code for bmm, keep in mind that Stan's HMC sampler requires continuous gradients to function correctly. Non-differentiable operations – such as `fabs()` or `fmax()` – create discontinuities in the log-density surface that can cause divergent transitions or break sampling entirely. Where such operations are mathematically necessary, bmm uses smooth approximations instead. For example, `sqrt(x^2 + epsilon)` serves as a differentiable stand-in for `|x|`. This is a standard Stan modelling technique and should be applied whenever your model's likelihood involves non-smooth functions.

Listing 3.1 R/bmm.R

```
bmm <- function(formula, data, model,
               prior = NULL,
               sort_data = getOption("bmm.sort_data", "check"),
               silent = getOption("bmm.silent", 1),
               backend = getOption("brms.backend", NULL),
               file = NULL, file_compress = TRUE,
               file_refit = getOption("bmm.file_refit", FALSE), ...) {
  deprecated_args(...)
  dots <- list(...)
  local_brms_threads(dots)
  file <- check_rds_file(file)
  file_refit <- validate_file_refit(file_refit)

  # check if the model has been previously fit and return it if requested.
  # "on_change" cannot be answered here -- it needs the Stan code and data that
  # only exist further down -- so it is checked where brms checks it, after
  # code and data generation and before compilation
  if (file_refit == "never") {
    x <- try_read_bmmfit(file)
    if (!is.null(x)) {
      return(x)
    }
  }

  # set temporary global options and return modified arguments for brms
  configure_opts <- nlist(
    sort_data, silent, backend,
    parallel = dots$parallel,
    cores = dots$cores
  )
  opts <- configure_options(configure_opts)
  dots$parallel <- NULL

  # check model, formula and data, and transform data if necessary
  user_formula <- formula
  model <- check_model(model, data, formula)
  data <- check_data(model, data, formula)
  formula <- check_formula(model, data, formula)

  # generate the model specification to pass to brms later
  config_args <- configure_model(model, data, formula)

  # configure the default prior and combine with user-specified prior
  frame_args <- brms_frame_args(dots)
  prior <- brms::do_call(32configure_prior, c(list(model, data, config_args$formula, prior), f

  # configure initial values; the prior decides which parameters exist
  config_args$init <- brms::do_call(
    create_initfun, c(list(model, data, config_args$formula, prior), frame_args)
  )
}
```

Listing 3.2 R/helpers-model.R

```
configure_model <- function(model, data, formula) {  
  UseMethod('configure_model')  
}
```

Listing 3.3 R/model_ezdm.R

```
.ezdm_version_table <- list(  
  "3par" = list(  
    parameters = list(...),  
    links = list(...),  
    fixed_parameters = list(...),  
    priors = list(...),  
    init_ranges = list(...)  
  ),  
  "4par" = list(  
    parameters = list(...),  
    links = list(...),  
    ...  
  )  
)
```

4 Example model file

All models in the package are defined as S3 classes and follow a strict template. This allows us to implement general methods for handling model fitting, data checking, and post-processing. Each model has an internal function that defines the model and its parameters, and a user-facing alias. Let's look at how two models are implemented - the IMM model, which uses both general class and specific model methods, but no custom stan code, and the SDM model, which depends heavily on custom stan code. If you use the `use_model_template()` function, templates for all sections below will be automatically generated for your model.

4.1 The Interference Measurement Model (IMM)

The model is defined in the file `R/model_imm.R`. Let's go through the different parts.

4.1.1 Model definition

The full IMM model is defined in the following internal model class:

Here is a brief explanation of the different components of the model definition:

resp_vars: a list of response variables that the model will be fitted to. These variables will be used to construct the `brmsformula` passed to `brms` together with the `bmmformula` and the `parameters` of the model. The user has to provide these variables in the data frame that is passed to the `bmm()` function

other_vars: a list of additional variables that are required for the model. This is used to check if the data contains all necessary information for fitting the model. In the example above, the IMM model requires the names of the variables specifying the non-target features relative to the target, the variables specifying the distance of the non-targets to the target, and the `set_size`. The user has to provide these variables in the data frame that is passed to the `bmm()` function

domain, task, name, citation, requirements: contains information about the model, such as the domain, task, name, citation, requirements. This information is used for generating help pages

version: if the model has multiple versions, this argument is specified by the user. Then it is used to dynamically adjust some information in the model object. In the case of the `imm` model, we have three versions - `full`, `bsc` and `abc`. As you can see at the end of the script, some parameters are deleted depending on the model version.

parameters: contains a named list of all parameters in the model that can be estimated by the user and their description. This information is used internally to check if the `bmmformula` contains linear model formulas for all model parameters, and to decide what information to include in the summary of `bmmfit` objects.

links: a named list providing the link function for each parameter. For example, `kappa` in the `imm` models has to be positive, so it is sampled on the log scale. This information is used in defining the model family and for the summary methods. The next to last line, `set_links()`, applies and validates links the user passes. The IMM builds its links into the mixture family and the non-linear formulas, so it declares `settable_links.imm()` returning `character(0)`, and any link a user passes is refused (see Section 3.3.1)

fixed_parameters in the `imm` several parameters are fixed to constant values internally to identify the model. Only one of them, `mu1` is also part of the `parameters` block - this is the only fixed parameters that users can choose to estimate instead of leaving it fixed. `mu2` and `kappa2` cannot be freely estimated.

default_priors a list of lists for each parameter in the model. Each prior has three components: **main**, the prior that will be put on the Intercept or on each level of a factor if the intercept is suppressed; **effects**, the prior to put on the regression coefficients relative to the intercept; and **sd**, the prior on the standard deviations of random effects. The priors are described as in the `set_prior` function from `brms`. This information is used by the `configure_prior()` S3 method to automatically set the default priors for the model. The priors that you put here will be used by `bmm()` unless the users chooses to overwrite them.

init_ranges: a range on the native scale for each parameter's initial values. The IMM's are the central 50% of each parameter's **main** prior; see the architecture chapter for how they are used.

regex: For the `imm` models, the `nt_features` and `nt_distances` variables can be specified with regular expressions, if the user sets `regex = TRUE`

call: this automatically records how the model was called so that the call can be printed in the summary after fitting. Leave it as is.

class: is the most important part. It contains the class of the model. This is used by generic S3 methods to perform data checks and model configuration. The classes should be ordered from most general to most specific. A general class exists when the same operations can be performed on multiple models. For example, the `'3p'`, `'imm_abc'`, `'imm_bsc'` and `'imm_full'` models all have `non-targets` and `set_size` arguments, so the same data checks can be performed on all of them, represented by the class `non_targets`. The first class should always be `bmmmodel`, which is the main class for all models. The last class should be the specific model name, in

this case `imm_full`, `imm_abc` or `imm_bsc`, which is automatically constructed if a `version` argument is provided. Otherwise the last class will be just the name of the model.

4.1.2 Model alias

The model alias is a user-facing function that calls the internal model function. It is defined as follows:

The details will be filled out automatically from the model definition. This does some fancy formatting to include documentation about all versions of the model in the same help file.

4.1.3 `check_data()` methods

Each model should have a `check_data.modelname()` method that checks if the data contains all necessary information for fitting the model. For the IMM, the `bsc` and `full` versions require a special check for the `nt_distances` variables:

The IMM models share methods with the `mixture3p` model, all of which are of class `non_targets` so the `check_data.non_targets` method is defined in the general file `R/helpers-data.R`. If you are adding a new model, you should check if the data requirements are similar to any existing model and define the `check_data` method only for the methods that are unique to your model.

The `check_data.mymodel()` function should always take the arguments `model`, `data`, and `formula` and return the data with the necessary transformations. It should also call `NextMethod("check_data")` to call the `check_data` method of the more general class.

To understand what validation your model inherits, trace the `NextMethod` dispatch chain for your class. For example, the `imm_full` model with class `c("bmmodel", "circular", "non_targets", "imm", "imm_full")` dispatches as:

1. `check_data.bmmodel` — generic checks (data is coercible to a data frame and has rows), then sets the `data_name` and `checked` attributes
2. $\rightarrow$ `NextMethod` $\rightarrow$ `check_data.circular` — validates response is in radians, wraps to $(-\pi, \pi]$
3. $\rightarrow$ `NextMethod` $\rightarrow$ `check_data.non_targets` — validates `nt_features`, `set_size`, creates index variables, stores data attributes
4. $\rightarrow$ `NextMethod` $\rightarrow$ no `check_data` method is defined for the `imm` class, so the chain passes straight through it
5. $\rightarrow$ `NextMethod` $\rightarrow$ `check_data.imm_full` — validates `nt_distances` (model-specific)

The order is **general first, specific last** — the reverse of what most people expect. S3 dispatch walks the class vector from left to right, and `"bmmmodel"` is the leftmost class, so `check_data.bmmmodel` runs *before* `check_data.imm_full`, not after. A class with no method for the generic is simply skipped.

This matters for two reasons. Your model-specific method runs last, so it can rely on everything the general methods have already done: by the time `check_data.imm_full` runs, the response has been wrapped to radians and `max_set_size` is already an attribute on the data. And it means a fall-through fallback must go on `.default`, not on `.bmmmodel` — see Chapter 6.

This also means a new circular model with non-targets automatically gets radian validation, non-target checks, and column existence checks — you only need to write methods for your model-specific validations.

4.1.4 `configure_model()` methods

The `configure_model.mymodel()` method is where you specify the model formula, the family, any custom code. The method is defined as follows for the IMM model:

(we show only the `IMMfull` version)

The three IMM versions differ only in the `theta` expression and in which extra non-linear terms they add, so everything they share lives in three private helpers at the top of `R/model_imm.R`:

The version-specific method then builds `theta_exprs` as a glue vector — one expression per non-target position — and hands it to `.add_imm_mixture_terms()`, which loops. `imm_abc` passes `"{lure_idx} * a + (1 - {lure_idx}) * (-100)"`, `imm_full` the distance-weighted expression above. This is the pattern to copy if your model has versions that share a formula skeleton: put the skeleton in a helper prefixed with a dot, and let each version supply only what differs. Note the dot prefix, but not `.model_` — `supported_models()` scans for that prefix and would list your helper as a model.

The `configure_model` method should always take the arguments `model`, `data`, and `formula` (as a `bmmformula`) and return a named list with the formula (as a `brmsformula`) and the data. The `brmsfamily` should be stored within the formula.

Inside the `configure_model` method the `brmsformula` is generated using the `bmf2bf` function. This function converts the `bmmformula` passed to `bmm()` function into a `brmsformula` based on the information for the response variables provided in the `bmmmodel` object. There is a general method in `R/bmmformula.R` to construct the formula for all models with a single response variable.

Read the dispatch order carefully here, because it is the reverse of what the names suggest. `bmf2bf()` is called with the model object, whose class vector starts with `"bmmmodel"`, so `bmf2bf.bmmmodel` runs **first**. Its `NextMethod("bmf2bf")` walks on to the rest of the chain, and only reaches `bmf2bf.default` if no model-specific method intercepted.

That is why `bmf2bf.default` returns `NULL` rather than a formula. It means “nobody claimed the first line”. `bmf2bf.bmmmodel` sees the `NULL`, falls through the `%||%`, and builds the single-response first line `resp ~ 1` itself. A model with several response variables, or one needing `vreal()`, `vint()` or `trials()`, defines `bmf2bf.mymodel` that returns the first line it wants; `NextMethod()` then returns that instead of `NULL` and the `%||%` fallback never fires.

Once the first line exists, the rest is mechanical: each parameter formula becomes `brms::nlf()` if it is marked non-linear and `brms::lf()` otherwise, and `Reduce()` adds them all to the base formula. Whether a component is marked non-linear is decided in `assign_nl_attr()` purely by whether one of its predictors is also a predicted parameter — which is the mechanism behind the name-clash warning described in the next chapter.

This conversion from a `bmmformula` object into a `brmsformula` object is done to avoid users having to specify complicated and long formulas or specifying all additional response information in the `brmsformula` themselves. For more detailed information on the use of additional response information in a `brmsformula` please see the [brmsformula documentation](#).

4.2 The Signal Discrimination Model (SDM)

The SDM model is defined in the file `R/model_sdm.R`. The SDM model differs in the configuration compared to the IMM model, as it requires custom STAN code. Let’s go through the different parts. As before, we start with the model definition.

4.2.1 Model definition

Like the IMM, the SDM sets `init_ranges`, which `create_initfun()` uses to generate starting values. Here they are hand-set rather than derived from the priors.

The model definition is similar to the IMM model, but the SDM model only requires the user to specify the response error, but not additional variables such as non-target variables. The `class` is also different, as the SDM model is not a subclass of the IMM model. We’ll skip the alias for the SDM model, as it is similar for every model.

4.2.2 `check_data()` methods

The SDM shares a class with other `circular` models, so most of the data checks are performed by `check_data.circular` method, defined in the general file `R/helpers-data.R`. The `sdm` however, samples much more quickly in Stan, if the data is sorted by the predictor variables, so we have the following custom data check method for the `sdm`:

This is a compact example of the attribute bridge. Having sorted the data, the method computes where each run of identical predictor values starts and how long it is — the information the

Stan likelihood needs to compute its normalizing constant once per run rather than once per row — and hangs the result on the data frame as `sdm_run_metadata`. `configure_model.sdm` reads it back out a few pipeline steps later. The computation itself is a separate function, `sdm_run_metadata()`, and that matters: it is the reason `configure_model.sdm` can recompute the metadata when it is called directly and the attribute is absent.

One detail in that function is worth copying. `brms` drops rows with missing values in any model variable before fitting, so run boundaries computed on the full data frame would not line up with the rows Stan actually sees. `sdm_run_metadata()` therefore applies `stats::complete.cases()` over the response and predictor columns before computing the runs. Any index you precompute in `check_data` and pass through to Stan has the same problem.

4.2.3 `configure_model()` methods

The `configure_model` method for the SDM model is different compared to the IMM model, as it requires custom STAN code. The method is defined as follows:

Note that `configure_model` no longer returns `init` — initial values are now handled separately by `create_initfun()`, which uses the `init_ranges` defined in the model object.

This is the longest `configure_model` method in the package, so it is worth separating the parts that every custom-family model needs from the parts that are specific to SDM.

Every custom-family model needs the first two. The `brms::custom_family()` call declares the parameters (`dpars`), their link functions, bounds, whether the likelihood is vectorized (`loop = FALSE` here), and the R functions that implement `log_lik`, `posterior_predict` and `posterior_epred`. For more information, see the [brms custom families vignette](#). And the Stan code is read from `inst/stan_chunks/` through `system.file()` — not a relative path — so that it is found once the package is installed rather than only when it is loaded from source. Each chunk becomes its own `brms::stanvar()`, tagged with the Stan block it belongs in.

The rest is SDM-specific, and all of it exists to make the normalizing constant cheap. Three things are going on:

- **Data passed to Stan, not just code.** `brms::stanvar(x = ..., name = ...)` passes an R *value* into the Stan data block. SDM passes the run count and, through the helper `sdm_stanvar_int_array()`, the run start and length arrays computed back in `check_data.sdm`. The helper exists because `stanvar()` would declare an integer array as `vector`, so it overwrites the generated `scode` with `array[G_sdm_runs] int <name>;`.
- **Two likelihood chunks.** Which one is read depends on `brms_slices_likelihoood()`. See the threading section of the architecture chapter for why there are two and what `threading(force = TRUE)` does.

- **pll_args**. Data that the threaded likelihood needs must also be declared in the partial-log-likelihood signature. That is what `pll_args = "data matrix COSN"` on the `tdata` stanvar does, and what `sdm_stanvar_int_array()` sets for the index arrays. Without it the variable is out of scope inside `partial_log_lik` and the model does not compile.

Converting the `bmmformula` to a `brmsformula` and collecting all arguments is done entirely using the `bmf2bf` method.

4.2.4 Post-processing methods

Unlike the `imm` model, the `sdm` model requires some special post-processing because of the way the link functions are coded. These methods are applied *after* the `brmsfit` object is returned, at the very end of the `bmm()` pipeline:

The `revert_postprocess_brm` method is the inverse of `postprocess_brm`. It is called internally when functions like `conditional_effects()` or `emmeans()` need to temporarily undo post-processing to work with the model in its original `brms` representation. If your `postprocess_brm` method modifies the fitted object (e.g., changing link functions, renaming parameters), you should also define a `revert_postprocess_brm` method that reverses those changes.

We also have a couple of special functions for custom families in `brms` (see the `log_lik` and `posterior_predict` argument in the call to `brms::custom_familiy()`), which allow other typical tools from `brms` such `posterior_predict` or `bridgesampling` to work:

4.3 The Diffusion Decision Model (DDM)

The DDM model is defined in `R/model_ddm.R`.

4.3.1 Model definition

It demonstrates several patterns that differ from the circular models above: suppressing `mu` for a model with no natural location parameter, `init_ranges` for MCMC initialization, and a custom `bmf2bf` method for response variable wrapping.

The DDM uses a defaults table to store parameters, links, priors, and `init_ranges` in one place:

Key differences from the circular models:

- **No natural `mu`**: The DDM has no location parameter. A dummy `mu` is included in the family's `dpars` and fixed to 0 via `fixed_parameters`, which satisfies `brms`' requirement that every custom family has a `mu` parameter.

- **Two response variables:** `resp_vars = nlist(rt, response)` — reaction time and binary choice. This requires a custom `bmf2bf` method.
- **init_ranges:** Specifies plausible starting ranges for each parameter to ensure stable MCMC sampling. The `create_initfun()` step uses these.
- **Short class chain:** `c("bmmodel", "ddm")` — no intermediate domain class. RT models don't share as many `check_data` patterns as the circular models do.

4.3.2 User-facing alias

The DDM alias follows the standard pattern — calling `stop_missing_args()` and then the internal constructor:

4.3.3 bmf2bf method

Because the DDM has two response variables (`rt` and `response`), it needs a custom `bmf2bf` method to construct the first line of the brms formula with the proper response wrapping:

This creates a formula like `rt | dec(response) ~ 1`, where `dec()` tells brms which variable codes the binary decision (0 = lower, 1 = upper boundary).

4.3.4 configure_model method

Note that:

- `mu` is included in `dpars` with an `identity` link and fixed to 0 (the `mu`-suppression pattern)
- `vars = "dec[n]"` passes the decision variable to the Stan likelihood
- The `log_lik` and `posterior_predict` functions use the `dddmm()` and `rddmm()` distribution functions from `R/distributions.R`
- `configure_model` returns `nlist(formula, data, stanvars)` without `init` — initialization is handled by `create_initfun()`

4.3.5 check_data method

The DDM has thorough data validation for RT and response variables:

The DDM's `check_data` is a good example of thorough validation: it checks variable existence, removes NAs with warnings, validates RT ranges, and flexibly accepts multiple response coding formats (integer, logical, character) while coercing them all to 0/1.

4.4 The EZ-Diffusion Model (EZDM)

The EZDM model (R/model_ezdm.R) demonstrates two additional patterns: the **version table** pattern for managing multiple model versions, and working with **aggregate-level data** (summary statistics rather than trial-level observations).

4.4.1 Version table

Instead of using if/else logic to handle version-specific parameters (as the IMM does), the EZDM uses a lookup table:

The model constructor then indexes into this table:

```
.model_ezdm <- function(mean_rt = NULL, var_rt = NULL, n_upper = NULL,
                        n_trials = NULL, version = "3par", links = NULL,
                        call = NULL, ...) {
  out <- structure(
    list(
      resp_vars = nlist(mean_rt, var_rt, n_upper),
      other_vars = nlist(n_trials),
      ...
      parameters = .ezdm_version_table[[version]][["parameters"]],
      links = .ezdm_version_table[[version]][["links"]],
      fixed_parameters = .ezdm_version_table[[version]][["fixed_parameters"]],
      default_priors = .ezdm_version_table[[version]][["priors"]],
      init_ranges = .ezdm_version_table[[version]][["init_ranges"]]
    ),
    class = c("bmmmodel", "ezdm"),
    call = call
  )
  if (!is.null(version)) class(out) <- c(class(out), paste0("ezdm_", version))
  ...
}
```

This pattern is cleaner and easier to extend than conditional logic. New models with multiple versions should prefer this approach.

4.4.2 bmf2bf for aggregate data

The EZDM operates on aggregate statistics (mean RT, variance of RT, number of upper responses, total trials) rather than trial-level data. This requires version-specific **bmf2bf** methods that pack multiple response variables into brms' special response terms:

This creates a formula like `mean_rt | vreal(var_rt) + vint(n_upper) + trials(n_trials)`
`~ 1`, where:

- `vreal()` passes real-valued additional response information to Stan
- `vint()` passes integer-valued additional response information
- `trials()` passes the trial count

These special brms terms make the additional variables available in the Stan likelihood via `vreal1[n]`, `vint1[n]`, and `trials[n]`.

4.4.3 Version-specific `configure_model`

Each EZDM version has its own `configure_model` method because the Stan likelihood functions differ:

Note the `vars` argument in `custom_family` — it lists the additional response terms that the Stan likelihood needs access to. The Stan code comes from `.ezdm_stan_functions()`, which joins chunks shared by both versions (`ezdm_series.stan`, `ezdm_cumulants.stan`) with the version's own functions; a helper like this keeps shared Stan code in one file.

We will now look at how to construct all these parts for a new model. **Hint:** you don't have to do it manually, you can use the `use_model_template()` function to generate templates for your model.

Listing 4.1 R/model_imm.R

```
.model_imm <- function(resp_error = NULL, nt_features = NULL, nt_distances = NULL,
                      set_size = NULL, regex = FALSE, version = "full", links = NULL,
                      call = NULL, ...) {
  out <- structure(
    list(
      resp_vars = nlist(resp_error),
      other_vars = nlist(nt_features, nt_distances, set_size),
      domain = "Visual working memory",
      task = "Continuous reproduction",
      name = "Interference measurement model by Oberauer and Lin (2017).",
      citation = glue(
        "Oberauer, K., & Lin, H.Y. (2017). An interference model \\
        of visual working memory. Psychological Review, 124(1), 21-59"
      ),
      version = version,
      requirements = glue(
        "- The response variable should be in radians and \\
        represent the angular error relative to the target
        - The non-target features should be in radians and be \\
        centered relative to the target"
      ),
      parameters = list(
        mu1 = glue(
          "Location parameter of the von Mises distribution for memory \\
          responses (in radians). Fixed internally to 0 by default."
        ),
        kappa = "Concentration parameter of the von Mises distribution",
        a = "General activation of memory items",
        c = "Context activation",
        s = "Spatial similarity gradient"
      ),
      links = list(
        mu1 = "tan_half",
        kappa = "log",
        a = "log",
        c = "log",
        s = "log"
      ),
      fixed_parameters = list(mu1 = 0, mu2 = 0, kappa2 = -100),
      default_priors = list(
        mu1 = list(main = "normal(0, 0.5)", effects = "normal(0, 0.25)", sd = "exponential(4)"),
        kappa = list(main = "normal(2, 1)", effects = "normal(0, 1)", sd = "exponential(1)"),
        a = list(main = "normal(0, 1)", effects = "normal(0, 1)", sd = "exponential(1)"),
        c = list(main = "normal(0, 1)", effects = "normal(0, 1)", sd = "exponential(1)"),
        s = list(main = "normal(0, 1)", effects = "normal(0, 1)", sd = "exponential(1)"),
      ),
      # central 50% of the main default prior on the native scale
      init_ranges = list(
        mu1 = c(-0.65, 0.65),
        kappa = c(3.8, 15),
        a = c(0.51, 2),
```

Listing 4.2 R/model_imm.R

```
#' @title `r .model_imm()$name`
#' @description Three versions of the `r .model_imm()$name` - the full, bsc, and abc.
#' `IMMfull()`, `IMMbsc()`, and `IMMabc()` are deprecated and will be removed in the future
#' Please use `imm(version = 'full')`, `imm(version = 'bsc')`, or `imm(version = 'abc')` instead.
#'
#' @name imm
#' @details `r model_info(.model_imm(), components = c('domain', 'task', 'name', 'citation'))`
#' ##### Version: `full`
#' `r model_info(.model_imm(version = "full"), components = c('requirements', 'parameters', 'f'))`
#' ##### Version: `bsc`
#' `r model_info(.model_imm(version = "bsc"), components = c('requirements', 'parameters', 'f'))`
#' ##### Version: `abc`
#' `r model_info(.model_imm(version = "abc"), components = c('requirements', 'parameters', 'f'))`
#'
#' Additionally, all imm models have an internal parameter that is fixed to 0 to
#' allow the model to be identifiable. This parameter is not estimated and is not
#' included in the model formula. The parameter is:
#'
#' - b = "Background activation (internally fixed to 0)"
#'
#' @param resp_error The name of the variable in the provided dataset containing
#' the response error. The response Error should code the response relative to
#' the to-be-recalled target in radians. You can transform the response error
#' in degrees to radian using the `deg2rad` function.
#' @param nt_features A character vector with the names of the non-target
#' variables. The non_target variables should be in radians and be centered
#' relative to the target. Alternatively, if regex=TRUE, a regular
#' expression can be used to match the non-target feature columns in the
#' dataset.
#' @param nt_distances A vector of names of the columns containing the distances
#' of non-target items to the target item. Alternatively, if regex=TRUE, a regular
#' expression can be used to match the non-target distances columns in the
#' dataset. Only necessary for the `bsc` and `full` versions.
#' @param set_size Name of the column containing the set size variable (if
#' set_size varies) or a numeric value for the set_size, if the set_size is
#' fixed.
#' @param regex Logical. If TRUE, the `nt_features` and `nt_distances` arguments
#' are interpreted as a regular expression to match the non-target feature
#' columns in the dataset.
#' @param version Character. The version of the IMM model to use. Can be one of
#' `full`, `bsc`, or `abc`. The default is `full`.
#' @param ... used internally for testing, ignore it
#' @return An object of class `bmmmodel`
#' @keywords bmmmodel
#' @examplesIf isTRUE(Sys.getenv("BMM_EXAMPLES"))
#' # load data
#' data <- oberauer_lin_2017
#'
#' # define formula
#' ff <- bmmformula(
#' kappa ~ 0 + set_size,
```

Listing 4.3 R/model_imm.R

```
#' @export
check_data.imm_bsc <- function(model, data, formula) {
  data <- .check_data_imm_dist(model, data, formula)
  NextMethod("check_data")
}

#' @export
check_data.imm_full <- function(model, data, formula) {
  data <- .check_data_imm_dist(model, data, formula)
  NextMethod("check_data")
}

.check_data_imm_dist <- function(model, data, formula) {
  nt_distances <- model$other_vars$nt_distances
  max_set_size <- attr(data, "max_set_size")

  stopif(
    !isTRUE(all.equal(length(nt_distances), max_set_size - 1)),
    "The number of columns for non-target distances in the argument \\"
    "'nt_distances' should equal max(set_size)-1})"
  )

  # replace NA values with 999 so they have 0 effect through the distance formula
  data[, nt_distances][is.na(data[, nt_distances])] <- 999

  stopif(
    any(data[, nt_distances] < 0),
    "All non-target distances to the target need to be postive."
  )
  data
}
```

Listing 4.4 R/model_imm.R

```
configure_model_imm_full <- function(model, data, formula) {  
  # retrieve arguments from the data check  
  max_set_size <- attr(data, "max_set_size")  
  lure_idx <- attr(data, "lure_idx_vars")  
  nt_features <- model$other_vars$nt_features  
  nt_distances <- model$other_vars$nt_distances  
  
  formula <- .base_imm_formula(model, formula) +  
    brms::nlf(theta1 ~ log(exp(c) + exp(a))) +  
    brms::nlf(expS ~ exp(s))  
  
  theta_exprs <- glue("{lure_idx} * log(exp(c-expS*{nt_distances}) + exp(a)) + (1 - {lure_idx} *  
  formula <- .add_imm_mixture_terms(formula, max_set_size, lure_idx, nt_features, theta_exprs)  
  formula$family <- .imm_mixture_family(max_set_size)  
  
  nlist(formula, data)  
}
```

Listing 4.5 R/model_imm.R

```
.base_imm_formula <- function(model, formula) {  
  bmf2bf(model, formula) +  
    brms::lf(kappa2 ~ 1) +  
    brms::lf(mu2 ~ 1) +  
    brms::nlf(kappa1 ~ kappa)  
}  
  
.add_imm_mixture_terms <- function(formula, max_set_size, lure_idx, nt_features, theta_exprs) {  
  kappa_nts <- paste0("kappa", 3:(max_set_size + 1))  
  theta_nts <- paste0("theta", 3:(max_set_size + 1))  
  mu_nts <- paste0("mu", 3:(max_set_size + 1))  
  
  for (i in 1:(max_set_size - 1)) {  
    formula <- formula +  
      glue_nlf("{kappa_nts[i]} ~ kappa") +  
      glue_nlf("{theta_nts[i]} ~ {theta_exprs[i]}") +  
      glue_nlf("{mu_nts[i]} ~ {nt_features[i]}")  
  }  
  
  formula  
}  
  
.imm_mixture_family <- function(max_set_size) {  
  brms::mixture(  
    brms::von_mises("tan_half"), brms::von_mises("identity"),  
    nmix = c(1, max_set_size),  
    order = "none"  
  )  
}
```

Listing 4.6 R/bmmformula.R

```
# If no model specific methods exist, it will construct a base brms formula from the first r
# We do it this way because most models require just one response variable
#' @export
bmf2bf.bmmmodel <- function(model, formula = bmmformula()) {
  brms_formula <- NextMethod("bmf2bf") %||% brms::bf(glue("{model$resp_vars[[1]]} ~ 1"))
  components <- lapply(formula, function(x) if (is_nl(x)) brms::nlf(x) else brms::lf(x))
  Reduce(`+`, components, init = brms_formula)
}

#' @export
bmf2bf.default <- function(model, formula) {
  NULL
}
```

Listing 4.7 R/model_sdm.R

```
.model_sdm <- function(resp_error = NULL, links = NULL, version = "simple", call = NULL, ...){
  out <- structure(
    list(
      resp_vars = nlist(resp_error),
      other_vars = nlist(),
      domain = "Visual working memory",
      task = "Continuous reproduction",
      name = "Signal Discrimination Model (SDM) by Oberauer (2023)",
      citation = glue(
        "Oberauer, K. (2023). Measurement models for visual working memory - \\
        A factorial model comparison. Psychological Review, 130(3), 841-852"
      ),
      version = version,
      requirements = glue(
        "- The response variable should be in radians and represent the angular \\
        error relative to the target"
      ),
      parameters = list(
        mu = glue("Location parameter of the SDM distribution (in radians; \\
        by default fixed internally to 0)"),
        c = "Memory strength parameter of the SDM distribution",
        kappa = "Precision parameter of the SDM distribution"
      ),
      links = list(
        mu = "tan_half",
        c = "log",
        kappa = "log"
      ),
      fixed_parameters = list(mu = 0),
      default_priors = list(
        mu = list(main = "normal(0, 0.5)", effects = "normal(0, 0.25)", sd = "exponential(4)"),
        kappa = list(main = "student_t(5, 1.75, 0.75)", effects = "normal(0, 1)", sd = "exponential(4)"),
        c = list(main = "student_t(5, 2, 0.75)", effects = "normal(0, 1)", sd = "exponential(4)"),
      ),
      init_ranges = list(
        mu = c(-0.5, 0.5),
        kappa = c(2.5, 3.5),
        c = c(4, 6)
      )
    ),
    class = c("bmmmodel", "circular", "sdm", paste0("sdm_", version)),
    call = call
  )
  out <- set_links(out, links)
  out
}
```

Listing 4.8 R/model_sdm.R

```
#' @export
check_data.sdm <- function(model, data, formula) {
  # data sorted by predictors is necessary for speedy computation of normalizing constant
  data <- order_data_query(model, data, formula)
  attr(data, "sdm_run_metadata") <- sdm_run_metadata(data, formula, model)
  NextMethod("check_data")
}
```

Listing 4.9 R/model_sdm.R

```
#' @export
configure_model.sdm <- function(model, data, formula) {
  # note - c has a log link, but I've coded it manually for computational efficiency
  sdm_simple <- brms::custom_family(
    name = "sdm_simple",
    dpars = c("mu", "c", "kappa"),
    links = c("tan_half", "identity", "log"),
    lb = c(NA, NA, NA),
    ub = c(NA, NA, NA),
    type = "real", loop = FALSE,
    log_lik = log_lik_sdm_simple,
    posterior_predict = posterior_predict_sdm_simple,
    posterior_epred = posterior_epred_sdm_simple
  )

  # prepare initial stanvars to pass to brms, model formula and priors
  sc_path <- system.file("stan_chunks", package = "bmm")
  stan_funs <- read_lines2(paste0(sc_path, "/sdm_simple_funs.stan"))
  stan_tdata <- read_lines2(paste0(sc_path, "/sdm_simple_tdata.stan"))
  likelihood_file <- if (brms_slices_likelihoood()) {
    "sdm_simple_likelihoood_threaded.stan"
  } else {
    "sdm_simple_likelihoood.stan"
  }
  stan_likelihoood <- read_lines2(paste0(sc_path, "/", likelihood_file))
  stan_tdata_pll_args <- if (brms_slices_likelihoood()) {
    "data matrix COSN"
  }
  run_metadata <- attr(data, "sdm_run_metadata")
  if (is.null(run_metadata)) {
    # Guard direct configure_model.sdm() calls that bypass check_data.sdm().
    run_metadata <- sdm_run_metadata(data, formula, model)
  }
  stanvars <- brms::stanvar(scode = stan_funs, block = "functions") +
    brms::stanvar(scode = stan_tdata, block = "tdata", pll_args = stan_tdata_pll_args) +
    brms::stanvar(x = run_metadata$G_sdm_runs, name = "G_sdm_runs") +
    sdm_stanvar_int_array(run_metadata$sdm_run_start, "sdm_run_start", "G_sdm_runs") +
    sdm_stanvar_int_array(run_metadata$sdm_run_count, "sdm_run_count", "G_sdm_runs") +
    brms::stanvar(scode = stan_likelihoood, block = "likelihoood", position = "end")

  # construct main brms formula from the bmm formula
  formula <- bmf2bf(model, formula)
  formula$family <- sdm_simple

  nlist(formula, data, stanvars)
}
```

Listing 4.10 R/model_sdm.R

```
#' @export
postprocess_brm.sdm <- function(model, fit, ...) {
  # manually set link_c to "log" since I coded it manually
  fit$family$link_c <- "log"
  fit$formula$family$link_c <- "log"
  fit
}

#' @export
revert_postprocess_brm.sdm <- function(model, fit, ...) {
  fit$family$link_c <- "identity"
  fit$formula$family$link_c <- "identity"
  fit
}
```

Listing 4.11 R/model_sdm.R

```
log_lik_sdm_simple <- function(i, prep) {
  mu <- brms::get_dpar(prep, "mu", i = i)
  c <- brms::get_dpar(prep, "c", i = i)
  kappa <- brms::get_dpar(prep, "kappa", i = i)
  y <- prep$data$Y[i]
  dsdm(y, mu, c, kappa, log = T)
}

posterior_predict_sdm_simple <- function(i, prep, ...) {
  mu <- brms::get_dpar(prep, "mu", i = i)
  c <- brms::get_dpar(prep, "c", i = i)
  kappa <- brms::get_dpar(prep, "kappa", i = i)
  rsdm(length(mu), mu, c, kappa)
}
```

Listing 4.12 R/model_ddm.R

```
.ddm_defaults <- list(
  parameters = list(
    drift = "Drift rate = Average rate of evidence accumulation of the decision processes",
    bound = "Boundary separation = Distance between the decision boundaries that need to be reached",
    ndt = "Non-decision time = Additional time required beyond the evidence accumulation process",
    zr = "Relative starting point = Starting point between the decision thresholds relative to the upper boundary"
  ),
  links = list(
    drift = "identity", bound = "log", ndt = "log", zr = "logit"
  ),
  fixed_parameters = list(
    zr = 0,
    mu = 0
  ),
  priors = list(
    drift = list(main = "cauchy(0,1)", effects = "normal(0,0.5)", sd = "exponential(1)"),
    bound = list(main = "normal(0,0.5)", effects = "normal(0,0.5)", sd = "exponential(2)"),
    ndt = list(main = "normal(-1.5,0.5)", effects = "normal(0,0.3)", sd = "exponential(2)"),
    zr = list(main = "normal(0,0.5)", effects = "normal(0,0.3)", sd = "exponential(2)")
  ),
  init_ranges = list(
    mu = c(-0.1,0.1),
    drift = c(-1,1),
    bound = c(1,2),
    ndt = c(0.01,0.05),
    zr = c(0.45,0.55)
  )
)

.model_ddm <- function(rt = NULL, response = NULL, links = NULL, call = NULL, ...) {
  out <- structure(
    list(
      resp_vars = nlist(rt, response),
      other_vars = nlist(),
      domain = "Decision Making / Response times",
      task = "Two-Alternative Force Choice RT",
      name = "Diffusion Decision Model",
      citation = glue(
        "Ratcliff, R. (1978). A theory of memory retrieval. Psychological Review, 85(2), 59-107."
      ),
      version = "NA",
      requirements = glue(
        "- The response time should be in seconds and \\
          represent the time between onset of the target stimulus until the response execution. \\
          - The response should be coded numerically: \\
          0 = lower response, 1 = upper response"
      ),
      parameters = .ddm_defaults[["parameters"]],
      links = .ddm_defaults[["links"]],
      fixed_parameters = .ddm_defaults[["fixed_parameters"]],
      default_priors = .ddm_defaults[["priors"]],
    )
  )
}
```

Listing 4.13 R/model_ddm.R

```
ddm <- function(rt, response, links = NULL, ...) {  
  call <- match.call()  
  stop_missing_args()  
  .model_ddm(rt = rt, response = response, links = links, call = call, ...)  
}
```

Listing 4.14 R/model_ddm.R

```
#' @export  
bmf2bf.ddm <- function(model, formula) {  
  rt <- model$resp_vars$rt  
  response <- model$resp_vars$response  
  brms::bf(paste0(rt, " | dec(", response, ") ~ 1"))  
}
```

Listing 4.15 R/model_ddm.R

```
#' @export
configure_model.ddm <- function(model, data, formula) {
  formula <- bmf2bf(model, formula)

  ddm_family <- function(link_drift, link_bound, link_ndt, link_zr) {
    brms::custom_family(
      "ddm",
      dpars = c("mu", "drift", "bound", "ndt", "zr"),
      links = c("identity", link_drift, link_bound, link_ndt, link_zr),
      lb = c(NA, NA, 0.1, 0, 0),
      ub = c(NA, NA, NA, NA, 1),
      type = "real",
      vars = "dec[n]",
      loop = TRUE,
      log_lik = log_lik_ddm,
      posterior_predict = posterior_predict_ddm,
      posterior_epred = posterior_epred_ddm
    )
  }

  formula$family <- ddm_family(
    link_drift = model$links$drift,
    link_bound = model$links$bound,
    link_ndt = model$links$ndt,
    link_zr = model$links$zr
  )

  sc_path <- system.file("stan_chunks", package = "bmm")
  stan_functions <- read_lines2(paste0(sc_path, "/ddm_functions.stan"))
  stanvars <- brms::stanvar(scode = stan_functions, block = "functions")

  nlist(formula, data, stanvars)
}
```

Listing 4.16 R/model_ddm.R

```
#' @export
check_data.ddm <- function(model, data, formula) {
  rt_var <- model$resp_vars$rt
  response_var <- model$resp_vars$response

  # check variables exist
  stopif(not_in(rt_var, colnames(data)),
        "The RT variable '{rt_var}' is not present in the data.")
  stopif(not_in(response_var, colnames(data)),
        "The response variable '{response_var}' is not present in the data.")

  # remove NAs with warning
  if (any(is.na(data[, rt_var]))) {
    data <- data[!is.na(data[, rt_var]), ]
    warning2("Some values in {rt_var} were NA. These were removed.")
  }

  # validate RT: must be positive, in seconds (warn if > 10s or < 0.1s)
  stopif(any(data[, rt_var] < 0, na.rm = TRUE),
        "Some reaction times are lower than zero.")
  warnif(any(data[, rt_var] > 10, na.rm = TRUE),
        "Your data contains reaction times larger than 10 seconds...")

  # validate response: accepts integer (0/1), logical, or character ("upper"/"lower")
  # coerces all formats to 0/1 integer
  # ... (full validation with type coercion)

  NextMethod("check_data")
}
```

Listing 4.17 R/model_ezdm.R

```
.ezdm_version_table <- list(
  "3par" = list(
    parameters = list(
      drift = "Drift rate = ...",
      bound = "Boundary separation = ...",
      ndt = "Non-decision time = ...",
      s = "The diffusion constant = ..."
    ),
    links = list(drift = "identity", bound = "log", ndt = "log", s = "log"),
    fixed_parameters = list(s = 0, mu = 0),
    priors = list(
      drift = list(main = "cauchy(0,1)", effects = "normal(0,0.5)"),
      bound = list(main = "normal(0,0.5)", effects = "normal(0,0.5)"),
      ndt = list(main = "normal(-1.5,0.5)", effects = "normal(0,0.3)"),
      s = list(main = "normal(0,1)", effects = "normal(0,0.3)")
    ),
    init_ranges = list(
      mu = c(0, 1), drift = c(-1, 1), bound = c(1, 2),
      ndt = c(0.25, 0.5), s = c(0.99, 1.01)
    )
  ),
  "4par" = list(
    parameters = list(...), # adds zr parameter
    links = list(...),      # adds zr = "logit"
    ...
  )
)
```

Listing 4.18 R/model_ezdm.R

```
#' @export
bmf2bf.ezdm_3par <- function(model, formula) {
  mean_rt <- model$resp_vars$mean_rt
  var_rt <- model$resp_vars$var_rt
  n_upper <- model$resp_vars$n_upper
  n_trials <- model$other_vars$n_trials

  brms::bf(paste0(mean_rt, " | vreal(", var_rt, ") + vint(", n_upper,
    ") + trials(", n_trials, ") ~ 1"))
}
```

Listing 4.19 R/model_ezdm.R

```
#' @export
configure_model.ezdm_3par <- function(model, data, formula) {
  # construct brms formula from the bmm formula
  formula <- bmf2bf(model, formula)
  links <- model$links

  # construct the family & add to formula object
  formula$family <- brms::custom_family(
    "ezdm_3par",
    dpars = c("mu", "drift", "bound", "ndt", "s"),
    links = c("identity", links$drift, links$bound, links$ndt, links$s),
    lb = c(NA, NA, 0, 0, 0),
    ub = c(NA, NA, NA, NA, NA),
    type = "real",
    log_lik = log_lik_ezdm_3par,
    posterior_predict = posterior_predict_ezdm_3par,
    posterior_epred = posterior_epred_ezdm_3par,
    loop = TRUE,
    vars = c("vreal1[n]", "vint1[n]", "trials[n]")
  )

  # prepare initial stanvars to pass to brms, model formula and priors
  stanvars <- brms::stanvar(scode = .ezdm_stan_functions("3par"), block = "functions")

  # return the list
  nlist(formula, data, stanvars)
}
```

5 Adding a new model

If you have read [Section 3](#) and [Section 4](#), you should have a pretty good idea of how the `bmm` package functions. Now it's time to add your new model.

You don't have to create the files by hand. `use_model_template()` generates the model file with stubs for every method a model needs, so you can focus on filling them in. The chapter ends with a checklist of everything a new model has to touch *outside* its own file, because the test suite fails on some of those and review catches the rest.

What `use_model_template()` does

Run it from the root of the package. It writes `R/model_<model_name>.R` containing the parameter specification, the model constructor, the user-facing alias, and stub methods for `check_data`, `bmf2bf`, `configure_model` and `postprocess_brm`. With `custom_family = TRUE` it also creates one empty `inst/stan_chunks/<model_name>_<block>.stan` file per entry in `stanvar_blocks`, and wires the code that reads them into `configure_model`.

The full argument list is on the [reference page for `use\_model\_template\(\)`](#). Three things about the call are worth knowing:

- **versions is the second positional argument.** `use_model_template("gcm", TRUE)` means `versions = TRUE`, not `custom_family = TRUE`. Name the arguments.
- **The stan files are created before the R file is written.** If the call fails partway through, empty `.stan` files are already on disk and you have to remove them by hand before you can call the function again.
- **An unversioned model gets `version = "NA"`** — the literal string, not `NA`.

What the template leaves for you

The generated file is a starting point that lags behind the package's conventions in a few places. Fix these before your first `check()`:

- **Add `#' @keywords bmmmodel`** to the alias's roxygen block. The pkgdown test fails without it under `devtools::test()`; it is skipped inside `check()`, so do not rely on `check()` to catch it (see [Chapter 6](#)).

- **Add an sd prior to every parameter.** The template emits only main and effects; see Section 3.8.
- **Add `log_lik`, `posterior_predict` and `posterior_epred` to the `brms::custom_family()` call.** The skeleton has none of them, and without them `loo()`, `pp_check()` and `posterior_epred()` fail on your model (step 8 below).
- **Add `...` to `postprocess_brm.<model>()`,** or delete the section. The generic is `postprocess_brm(model, fit, ...)` and the stub has only two arguments.
- **Uncomment or delete the `settable_links` stub, all of it.** If you leave it as generated, roxygen attaches its `#' @exportS3Method` line to the alias below it.

There are no stubs for `check_formula()`, `configure_prior()`, `create_initfun()`, `revert_check_data()` or the optional post-fit methods; add those only if your model needs them.

5.1 Example

Let's add a new model called `gcm`. Assume you have tested the model in Stan and have the Stan code ready. We want a custom family with two Stan blocks, `likelihood` and `functions` (see `?brms::stanvar` for the blocks).

First set up your system and git environment as described in Section 1. Then load the development version with `devtools::load_all()` and run, from the root directory of the `bmm` package:

```
use_model_template("gcm", custom_family = TRUE, stanvar_blocks = c("likelihood", "functions"))
```

This creates `R/model_gcm.R` and the empty files `inst/stan_chunks/gcm_likelihood.stan` and `inst/stan_chunks/gcm_functions.stan`, and opens them in your editor. The top of the R file holds the specification and the constructor:

```
.gcm_defaults <- list(
  parameters = list(
    par1 = "Parameter 1 = description of parameter 1",
    par2 = "Parameter 2 = description of parameter 2"
  ),
  links = list(
    par1 = "identity",
    par2 = "log"
  ),
  fixed_parameters = list(
    mu = 0
  )
)
```

```

),
priors = list(
  par1 = list(main = "normal(0, 1)", effects = "normal(0, 0.5)"),
  par2 = list(main = "normal(0, 0.5)", effects = "normal(0, 0.5)")
),
init_ranges = list(
  par1 = c(-1, 1),
  par2 = c(0.5, 1.5)
)
)

.model_gcm <- function(resp_var1 = NULL, required_arg1 = NULL, required_arg2 = NULL,
                      links = NULL, call = NULL, ...) {
  out <- structure(
    list(
      resp_vars = nlist(resp_var1),
      other_vars = nlist(required_arg1, required_arg2),
      domain = "",
      task = "",
      name = "",
      citation = "",
      version = "NA",
      requirements = "",
      parameters = .gcm_defaults[["parameters"]],
      links = .gcm_defaults[["links"]],
      fixed_parameters = .gcm_defaults[["fixed_parameters"]],
      default_priors = .gcm_defaults[["priors"]],
      init_ranges = .gcm_defaults[["init_ranges"]]
    ),
    class = c("bmmodel", "gcm"),
    call = call
  )
  out <- set_links(out, links)
  out
}

# uncomment if configure_model() builds the links into the family or into
# the non-linear formulas rather than reading them from the list above,
# so that a link set by the user is refused instead of silently ignored:
#' @exportS3Method
# settable_links.gcm <- function(model) character(0)

```

Below that come the user-facing alias `gcm()` with its roxygen block, and the stubs `check_data.gcm()`, `bmf2bf.gcm()`, `configure_model.gcm()` (with a `custom_family()` skeleton and the lines that read your two Stan files) and `postprocess_brm.gcm()`. Each stub except `configure_model` carries a header comment saying when you may delete it.

For a complete model written to the current conventions, read `R/model_sdt_yn.R`: a custom family on aggregated counts, with its own `log_lik`, `posterior_predict` and `posterior_epred` functions and a `settable_links` method. Most of its length is documentation.

5.2 Filling in the template

1. **The specification and the constructor.** Most of your work is in `.gcm_defaults`: the parameters (with descriptions), their links, which parameters are fixed and to what value, the default priors, and the initialization ranges. Set default priors (`main`, `effects` and `sd`) for every parameter, informed by knowledge in the field; `.ddm_defaults` in `R/model_ddm.R` is a worked example. Give `init_ranges` only to parameters whose likelihood is undefined in part of their range, such as a non-decision time that must stay below the fastest RT (see Section 3.10). Prefer parameter names of more than one letter that do not match a likely data column (see [Failure modes to expect](#)).

In `.model_gcm()`, rename the response and required arguments and fill in the metadata (`domain`, `task`, `name`, `citation`, `requirements`). Two things about the constructor are checked by tests: it must run with **no arguments** (the tests call every `.model_*` bare, which is why every argument defaults to `NULL`), and its fields must stay in the **order the template emits them**. For `domain`, reuse the string of the models your model sits with, so that `supported_models()` and the website list it under the right task group: "Visual working memory", "Working Memory (categorical)", "Categorical Decision Making", "Perception & Recognition Memory" (or "Recognition Memory", which lands in the same group) or "Decision Making / Response times". A new string becomes a group of its own.

For a model with several versions, generate the scaffold with `use_model_template("gcm", versions = c("v1", "v2"))`. You get a `.gcm_version_table` with one entry per version, a constructor that reads each field from it and appends `paste0("gcm_", version)` to the class, and an alias with `version = c("v1", "v2")` validated by `match.arg()`. Keep the default as a literal `c(...)`: `model_versions()` reads it to build the per-version documentation tables.

2. **Links.** The constructor passes the user's links through `set_links()`, which refuses names that are not parameters, repairs one-letter typos with a warning, and warns when a link widens the parameter's range (say `identity` instead of `log`). By default every link in your `links` list can be changed by the user. If `configure_model()` writes a link into

the family, the Stan code or the non-linear formulas instead of reading it from the list, a user's choice would be silently ignored, so declare which links can be set:

```
#' @exportS3Method
settable_links.gcm <- function(model) character(0) # or c("par1")
```

Name the method for your model class, not for `bmmodel`, which comes first in every class vector and would shadow it. A model that substitutes the inverse link into its formulas itself also overrides `settable_link_functions()`; the multinomial SDT models are the examples.

3. **The user-facing alias.** Rename the required arguments, fill in `@examples` with a full example, and add `@keywords` `bmmodel`. Title and details are generated from the constructor.
4. **`check_data.gcm()`** checks the data and returns it, ending with `NextMethod("check_data")`. It may compute transformations or store values as data attributes for `configure_model` (see Section 3.6). If it adds, renames or packs columns, `update()` needs to undo that before it can re-run it on a fitted model's data; see Section 3.7.
5. **`bmf2bf.gcm()`, if needed.** You need a custom method if your model has several response variables (DDM: `rt | dec(response) ~ 1`), uses aggregate data through `vreal()`, `vint()` or `trials()` (EZDM, `sdt_yn`), or needs other response wrapping. `brms` reads this formula as the linear model for the family's μ . If your model has no natural μ , suppress it: put `mu = 0` in `fixed_parameters`, add a dummy μ (identity link) to the family's `dpars`, and write the first formula line yourself (see Section 3.9). Leave the dummy μ out of `parameters`, so that `prior_info()` and `parameter_info()` do not report it.
6. **`configure_model.gcm()`** builds the formula, the family and the `stanvars` and returns `nlist(formula, data, stanvars)`. It does not return initial values. `brms` connects the family to your Stan code by name: a custom family "gcm" with `dpars = c("mu", "par1", "par2")` calls a Stan function `gcm_lpdf(y | mu, par1, par2)` (`gcm_lpmf` for an integer response) that you define in `gcm_functions.stan`. The SDM and DDM methods in Chapter 4 show the whole pattern, and the [brms custom families vignette](#) explains it. Add further Stan files to `inst/stan_chunks/` and read them here. If your Stan code has to differ when the user asks for within-chain threading, see Section 3.11.
7. **`postprocess_brm.gcm()`** changes the fitted object (renaming parameters and the like). Delete the section if you need none.
8. **`log_lik`, `posterior_predict` and `posterior_epred`**, for a custom family. These are plain R functions passed to `brms::custom_family()`; their contracts are in Chapter 6. They power LOO and WAIC, posterior predictive checks, and `posterior_epred()/fitted()`. Build them on `d*()` and `r*()` functions for your model in `R/distributions.R`, which you write first (see [Distribution functions](#)). If the mean of

your response is not a useful quantity, pass `posterior_epred_undefined("gcm")`, as SDM does, so the user gets an error that points to `native_parameters()`. Models built on a native brms family (`brms::mixture()`, `brms::multinomial()`) get all three from brms.

5.3 Before you open the pull request

Everything below lives outside your model file.

The test suite fails without these:

- An entry in `data_column_roles` (`R/helpers-model.R`) labelling each alias argument without a default: what the data column holds, in a few words. Use `<model>_<version>` keys where versions differ, and `NA` for an argument that names no column. Copy the entry of a similar model. The *Get started* article builds its model tables from it.
- A case in `stored_frame_cases()` in `tests/testthat/test-update.R` (a model, a formula and a small dataset, like the cases already there), and a `revert_check_data()` method if your `check_data()` changes columns (Section 3.7).
- Your model working in the loop of "`check_data()` returns a `data.frame()`" in `tests/testthat/test-helpers-data.R`. The loop calls every constructor with a shared set of arguments and a shared data frame; add a branch for your model if those do not fit it, as `sdt_ranking` and `sdt_cdp` have, or exclude it with a pointer to its own test, as `sdt_rating` is.
- `@keywords` `bmmmodel` on the alias, and a constructor that runs without arguments and keeps the template's field order.

Review expects these:

- A branch in `response_annotations()` (`R/helpers-model.R`) giving the coding each response variable needs ("radians in $[-\pi, \pi]$ ", "seconds"). `print()` shows it next to each response.
- A `domain` that places the model in the right task group (step 1).
- A test file, a parameter recovery study, an example dataset, an article, and a `NEWS.md` entry (below and Chapter 6).

Optional extension points. The defaults cover most models; implement one only when its condition applies.

Method	Implement when	Example
<code>pp_observables()</code> , <code>pp_simulate()</code>	the likelihood has more than one observable (RT and choice, counts and means), so <code>pp_check()</code> should check each. Add the model to <code>registered_models</code> in <code>test-pp_observables.R</code> .	<code>R/model_ddm.R</code>
<code>native_transform()</code>	the map from sampling to native scale is not elementwise <code>links</code> plus at most one softmax group, or depends on the design	<code>native_transform.non_targets()</code>
<code>data_check_findings()</code>	users commonly make a data mistake that <code>check_data()</code> tolerates; <code>bmm_data_check()</code> reports it	<code>data_check_findings.circular()</code>
<code>create_initfun()</code> , <code>configure_prior()</code>	<code>init_ranges</code> and <code>default_priors</code> cannot express what the model needs	<code>create_initfun.m3()</code>

A model fitted to aggregated counts can also be added to `uses_aggregate_data()` in `R/data_check.R` (currently `m3` and `ezdm`), so that `bmm_data_check()` does not note too few observations per cell.

5.4 Failure modes to expect

`configure_prior()` errors if `fixed_parameters` names a parameter the formula never gets. Every name in `fixed_parameters` has to be wired into the formula by your `configure_model()`, as either a distributional parameter or a non-linear one. This is the error you get when you add `mu = 0` to `fixed_parameters` but forget the dummy `mu` in the family's `dpars`.

Short parameter names are risky. `create_initfun()` maps Stan parameter names back to your model parameters with the regex `(~|_)param(_|$)`, taking the longest match. A parameter called `s` therefore matches `b_s_Intercept` but not `b_sim_Intercept`. A name that is a substring of another is safe; a name that appears as a whole underscore-delimited token elsewhere in the Stan parameter names is not.

A predictor that is also a parameter name and also a data column warns. `bmm` decides whether a formula component is non-linear by whether one of its predictors is also a predicted parameter. When the same name is a data column too, that choice changes the

likelihood without the user seeing it, so `check_formula()` warns. One-letter parameters collide with columns like a condition coded `c`; another reason to avoid them.

Method signatures follow the generics. `bmm()` calls `create_initfun()` with the prior as a fourth argument, so a method written as `function(model, data, formula)` fails on every fit. `configure_prior()` methods receive `data2`, `knots` or `drop_unused_levels` through ... when the user passes them, so a method without ... fails only then, which is the case nobody tests. R CMD check flags both. The full table is in Chapter 3.

5.5 Testing

Test your model with `testthat`; `use_test()` creates the file. `tests/testthat/test-model_ddm.R` and `test-model_sdt_yn.R` show how models are tested. Fitting takes too long for the test suite, so pipeline tests use brms' mock backend. `backend = "mock"` is a `brms::brm()` argument, and `mock_fit` reaches brms' mock backend through `brm()`'s ..., just as both reach `brm()` through `bmm()`'s ...: whatever you pass as `mock_fit` is returned in place of the fitted Stan object. Everything before the fit — data checks, formula translation, family construction, stanvars, priors, initial values — runs for real, which is what you want to test:

```
test_that("gcm model runs without errors", {
  withr::local_options("bmm.silent" = 2)
  skip_on_cran()

  dat <- data.frame(
    resp = rnorm(10),
    exemplar = rep(c("A", "B"), 5),
    category = rep(c(1, 2), each = 5)
  )

  f <- bmmformula(par1 ~ 1, par2 ~ 1)
  mock_fit <- bmm(
    f, dat,
    gcm(resp_var1 = "resp", required_arg1 = "exemplar", required_arg2 = "category"),
    backend = "mock", mock_fit = 1, rename = FALSE
  )
  expect_equal(mock_fit$fit, 1)
})
```

Two things about that call are easy to get wrong. The alias calls `stop_missing_args()`, so every argument without a default has to be supplied. And the argument is `mock_fit`. Older test files write `mock = 1`, which works only because R partial-matches it to `mock_fit` on the way through

`tests/testthat/helper-epred.R` provides `epred_vs_predict()` and `expect_epred_by_cell()` for testing a `posterior_epred` function against the mean of `posterior_predict`. If your Stan code switches on threading, test the generated code with and without `threads`, as `test-model_sdm-stancode.R` does.

The tests run with every `check()`. Make sure all of them pass before you submit.

5.5.1 Parameter recovery

Also run the model in a separate script on simulated data and check that it recovers the parameters: the hyper-parameters (means and standard deviations) and the subject-level parameters, with enough conditions to tell users how much data they need. Chapter 6 covers what the study has to show, in particular why you must not simulate and fit with the same distribution function, and how to report it.

5.5.2 Distribution functions

A custom family also needs distribution functions in `R/distributions.R`: `d*` and `r*`, optionally `p*` and `q*`. They are used to simulate data, inside `log_lik`, `posterior_predict` and `posterior_epred`, and for plotting.

5.6 Post-processing infrastructure

Most of the post-fit tools read the model object and need no code from you:

- `conditional_effects()` and the `emmeans` integration work through the `bmmfit` class.
- `native_parameters()` returns posterior draws of the parameters on their native scale, using your links through the default `native_transform()` method.
- `parameter_info()` and `prior_info()` read `parameters`, `fixed_parameters`, `links` and the default priors. A parameter in `fixed_parameters` but not in `parameters` (a dummy `mu`) is treated as family machinery and left out of the `prior_info()` report.
- `posterior_epred()` and `fitted()` call the `posterior_epred` function of your family. Models with class `"circular"` refuse them, because the mean of a circular error is not useful.
- `pp_check()` checks the brms response by default; `pp_observables()` extends it to further observables.
- `bmm_data_check()` runs your `check_data()` without compiling and adds anything your `data_check_findings()` method reports.

5.7 Add an example dataset

Every new model needs an example dataset that users can load and that the examples use. It can be a new dataset added to the package, one already in the package, or one from another package loaded with `data()`. The `mixture2p` and `mixture3p` articles use `data('bays2009_full', package='mixtur')`; the IMM and the SDT models use datasets shipped with `bmm`. For how to add one, see [R Packages](#).

5.8 Add an article

Every new model needs an article explaining the model and how to fit it with `bmm`. Create it with `use_article()` (see [R Packages](#)); the existing ones are in `vignettes/articles/`, and the published versions are [here](#). The README, the home page and the *Get started* tables list your model automatically, from its constructor and its `data_column_roles` entry.

Then submit your pull request.

6 Coding conventions

This chapter is the reference for how bmm code is written. It exists because your pull request will be judged against these conventions, and until now they lived mainly in the repository's `AGENTS.md`, which an outside contributor has no particular reason to open.

Read it as conventions, not as a style war. Most of the entries have a reason attached, and the reason is usually that the alternative broke something.

6.1 Errors, warnings and messages

Use the package's own helpers rather than the base functions:

Instead of	Use	Why
<code>stop()</code>	<code>stop2()</code>	Interpolates with glue, and suppresses the “Error in f(x):” call prefix so the user reads the message, not our internals
<code>warning()</code>	<code>warning2()</code>	Same
<code>message()</code>	<code>message2()</code>	Same, and respects <code>options(bmm.silent)</code>
<code>if (cond)</code>	<code>stopif(cond, ...)</code>	One line, and reads as the condition it guards
<code>stop2(...)</code>		
<code>if (cond)</code>	<code>warnif(cond, ...)</code>	Same
<code>warning2(...)</code>		
<code>sprintf()</code> in a message	<code>glue::glue()</code>	Named interpolation, no positional arguments to miscount
<code>paste(x, collapse = ", ")</code>	<code>collapse_comma(x)</code>	Also quotes each element

The base functions survive almost only inside the definitions of the helpers themselves and in `R/brms-misc.R`, which is vendored code: a copy of brms internals kept inside bmm rather than called through `brms::`, and marked read-only. `sprintf()` is still the right tool for formatting numbers (`"%.2f"`); the rule is about building messages.

Two of the reasons deserve spelling out, because they are not stylistic.

`message2()` checks `getOption("bmm.silent", 1)` and prints nothing at silence level 2 or above. Tests set `withr::local_options("bmm.silent" = 2)` to keep output clean. A base `message()` call ignores that and will make somebody's test noisy.

`stopif()` and `warnif()` pass `env.frame = -2` down to `stop2()`. That number counts frames back up the call stack: `-1` would interpolate in `stopif()`'s own frame, where your variables do not exist, so `-2` reaches past it into the function that called `stopif()`. This is what lets the message interpolate variables from *your* function. So this works and needs no `glue()` call of its own:

```
stopif(
  !version %in% c("custom", "cs", "ss"),
  'Unknown version: {version}. It should be one of "ss", "cs" or "custom"'
)
```

Write messages as one glue string. `stop2()` handles multi-line strings with trailing `\\` continuations, so a long message does not need `paste0()`.

6.2 Dependencies

Package code is **base R plus the native pipe** `|>`. `bmm` requires `R >= 4.1.0`, so `|>` is always available and `%>%` is not used in package code.

The Imports in DESCRIPTION are a short list, and adding to it needs a reason the reviewer will accept. **dplyr**, **tidyr**, **ggplot2** and **stringr** are **Suggests**, not **Imports** — `dplyr` and `tidyr` became optional in 1.2.0, and `magrittr` was removed in 1.3.0. If you have seen `bmm` guidance that prescribes `dplyr` idiom for package code, it is out of date. Tidyverse packages belong in articles and tests, where they are already available.

A function from a **Suggests** package must be guarded, or the package breaks for anyone who installed it without that suggestion. The pattern is:

```
stopif(
  !requireNamespace("tidyr", quietly = TRUE),
  'The "tidyr" package is required for this functionality'
)
data <- tidyr::gather(data, non_target_name, non_target_value, eval(nt_features))
```

as in `R/helpers-data.R`. Note the same function opens with

```
y <- y_nt <- non_target_name <- non_target_value <- NULL
```

which exists to silence R CMD check’s “no visible binding for global variable” notes from the non-standard evaluation in the `gather()` call. If you write code with bare column names, you need this line.

Always namespace explicitly: `brms::stanvar()`, `stats::complete.cases()`, `methods::formalArgs()`. `bmm` imports few functions directly, and an unqualified call to an Imports package is the kind of thing that works on your machine and fails in R CMD check.

6.3 Code shape

Return the last expression; use `return()` only to leave early. Guard clauses are the endorsed use:

```
x <- try_read_bmmfit(file, file_refit)
if (!is.null(x)) {
  return(x)
}
```

Do not write `return(out)` as the last line of a function.

No `set.seed()` inside a function, and no seed argument. A function that sets the seed silently rewrites the caller’s random state. `set.seed()` appears in `bmm` only in `@examples` blocks in the roxygen comments — the `#` comments above each function from which `devtools::document()` generates the help pages — where it is the right thing. If you need reproducibility inside a function, take the draws you were given; if you need it in a test, set the seed in the test.

snake_case for functions and variables. Private helpers are prefixed with a dot (`.base_imm_formula`, `.imm_mixture_family`). One reservation: a private helper must **not** be named `.model_*`, because `supported_models()` lists every object matching `~\.model_` as a model.

Comments say why, not what. `# construct the family` above a `brms::custom_family()` call adds nothing. The comment in `configure_model.sdm()` — `# note - c has a log link, but I've coded it manually for computational efficiency` — is worth its line, and so is every comment that records a constraint somebody would otherwise remove.

6.4 S3 conventions

Dispatch runs general to specific. A model's class vector starts with `"bmmodel"` (the class chains are listed in Section 3.4), so `check_data.bmmodel` runs *before* `check_data.sdm`, not after. This is the opposite of the intuition most people bring, and it has one sharp corollary: **a fall-through fallback belongs on `.default`, never on `.bmmodel`.** A method defined on `.bmmodel` runs first and shadows nothing, but a *fallback* placed there would be reached before the model-specific method had a chance to run. `bmf2bf` is the worked example — `bmf2bf.default` returns `NULL` and `bmf2bf.bmmodel` supplies the fallback after `NextMethod()` has come back empty (see Chapter 4).

Every `check_data` method ends with `NextMethod("check_data")`. Each method does its own model-specific validation and then hands on down the chain. The only method without it is `check_data.default`, which is the end of the chain and returns the data. Forget it in a method on a shared class (`circular`, `non_targets`, `sdt`) and every more specific method below it never runs, so a model's own checks are silently skipped. In a model's own method the omission does no harm today, but the next shared class added above it would break.

Match the generic's signature exactly, including `...` where the generic has one and the defaults on `check_model(model, data = NULL, formula = NULL)`. See the generics table in Chapter 3.

6.5 Where to validate arguments

Validate in the **exported constructor only** — the user-facing alias. `.model_*`(), `configure_model.*()` and private helpers should not, because their error messages name arguments the user never typed. The alias is the one place where the argument the message refers to is the argument the user actually passed:

Listing 6.1 R/model_m3.R

```
m3 <- function(resp_cats, num_options, choice_rule = "softmax", version = "custom", ...) {  
  call <- match.call()  
  stop_missing_args() # errors naming any argument with no default that the user omitted  
  stopif(  
    length(num_options) != length(resp_cats),  
    "The option variables should have the same length as the response variables."  
  )  
  .model_m3(...)  
}
```

`check_data.*()` is the exception, and not really an exception: it validates properties of the *data*, which the constructor cannot see. “Your `nt_distances` columns contain negative values” is a data message and belongs there.

6.6 `log_lik`, `posterior_predict` and `posterior_epred`

These are **plain functions**, not **S3 methods**. Do not name them `log_lik.mymodel` — that would register them against `brms`’ `log_lik` generic, which is not what you want. Name them `log_lik_<family_name>` and pass them to `brms::custom_family()`:

```
sdm_simple <- brms::custom_family(  
  name = "sdm_simple",  
  dpars = c("mu", "c", "kappa"), # distributional parameters  
  ...  
  log_lik = log_lik_sdm_simple,  
  posterior_predict = posterior_predict_sdm_simple,  
  posterior_epred = posterior_epred_sdm_simple  
)
```

`brms` checks the first arguments when you build the family: `log_lik` must start with `(i,` `prep)` and `posterior_predict` with `(i, prep, ...)`.

Both take `i`, the index of one observation, and `prep`, an object `brms` assembles from the fitted model and hands to you. `prep` holds the posterior draws and the data, and you pull values out of it rather than computing them: `brms::get_dpar()` for a parameter, `prep$data$Y` for the response.

The contract is easy to get wrong in one specific way. `brms::get_dpar(prep, "par", i = i)` returns a **vector with one value per posterior draw**, not a scalar, and your function must return a vector of that same length. The response is a scalar, `prep$data$Y[i]`:

Listing 6.2 R/model_sdm.R

```
log_lik_sdm_simple <- function(i, prep) {  
  mu <- brms::get_dpar(prep, "mu", i = i)  
  c <- brms::get_dpar(prep, "c", i = i)  
  kappa <- brms::get_dpar(prep, "kappa", i = i)  
  y <- prep$data$Y[i]  
  dsdm(y, mu, c, kappa, log = T)  
}
```

A density function that collapses its arguments (an `if` on a vector, a `sum()`) will return a scalar here, and `brms` will recycle it across all draws without complaint. The posterior predictive checks then look suspiciously clean. Check `length()` of what you return.

`posterior_epred_<family_name>` has a different contract: it takes only `prep`, is called once for all observations, and returns a **draws × observations matrix** of expected responses. `prep$dpars` then holds one draws × observations matrix per parameter, and data columns have to be lined up with them. Two helpers in `R/helpers-postprocess.R` do the bookkeeping: `.epred_data()` repeats a data column per draw, and `.epred_matrix()` shapes the result:

Listing 6.3 `R/model_sdt_yn.R`

```
posterior_epred_sdt_yn <- function(prepare) {  
  stimulus <- .epred_data(prepare$data$vint1, prepare)  
  dist <- .sdt_dist_names[prepare$data$vint2[1]]  
  p_old <- with(prepare$dpars, exp(.sdt_log_p_old(.sdt_eta(d, criterion, stimulus, sdratio), dist))  
  .epred_matrix(.epred_data(prepare$data$trials, prepare) * p_old, prepare)  
}
```

Pass it to `custom_family()` like the other two. A custom family without one fails in `posterior_epred()` with “object ‘posterior_epred_’ not found”, because `brms` looks for the function in the global environment. If the expected response of your model is not meaningful, pass `posterior_epred_undefined("<model>")`, which fails with a message pointing to `native_parameters()`.

`loop` in `custom_family()` is about the generated *Stan* code, not about this R function. With `loop = FALSE` `brms` emits a vectorized likelihood call, and with `loop = TRUE` it emits one call per observation. The consequence you will hit concerns `vars`, the `custom_family()` argument that names extra *Stan* variables your likelihood needs beyond the parameters (aggregate counts, trial numbers). With `loop = FALSE` you may not index those with `[n]`, because there is no per-observation loop to index inside, and `brms` stops with “Invalid use of index ‘[n]’ in an unlooped custom likelihood”. *SDM* uses `loop = FALSE` and no `vars`; *EZDM* uses `loop = TRUE` with `vars = c("vreal1[n]", "vint1[n]", "trials[n]")`.

6.7 Naming traps

Each of these cost somebody real debugging time, and none of them produce an error message that points at the cause.

A custom family’s `dpars` may not contain `_` or `.`, and may not end with a digit. Both are checked in `brms::custom_family()` (`brms` 2.23.0): “Dots or underscores are not allowed in ‘dpars’” and “‘dpars’ should not end with a number”. So neither `delta_1` nor `delta1` is a legal

dpar name. The digits you see on IMM parameters (`kappa1`, `theta3`, `mu2`) are generated by `brms::mixture()` itself, not declared by `bmm`, and non-linear parameters introduced through `nlf()` are not subject to this check.

Reusing a brms dpar name silently inherits its default prior. `brms:::def_dpar_prior()` attaches a default prior by dpar name (the three colons reach an unexported function, which is why you will not find it in brms' documentation), and the names it recognizes are `mu`, `sigma`, `shape`, `nu`, `phi`, `kappa`, `beta`, `zi`, `hu`, `zoi`, `coi`, `bs`, `ndt`, `bias`, `quantile`, `xi`, `alpha`, `disc` and `theta`. Worse, the lookup goes through `brms:::dpar_class()`, which strips trailing digits, so `kappa1` is looked up as `kappa`. `bmm` reuses several of these names deliberately and sets its own defaults over the top, which is fine — the trap is a parameter you reuse and then *forget* to give a default prior, which will quietly get brms' prior for an unrelated quantity.

A dpar must not be uniquely prefixed by thres. `brms:::stan_log_lik_custom()` assigns `p$thres` only for ordinal families but reads `p$thres` unconditionally when it builds the likelihood call. `p` is a list keyed by dpar name, and `$` partial-matches on lists, so a dpar named `threshold` is found by `p$thres` and passed where the ordinal thresholds belong. The result is Stan code that does not parse, from a name that looks perfectly reasonable.

Stan probability-function suffixes require the bar syntax at every call site. `stanc`, the Stan compiler, says it plainly (version 2.39):

Probability functions with suffixes `_lpdf`, `_lupdf`, `_lpmf`, `_lupmf`, `_cdf`, `_lcdf` and `_lccdf`, require a vertical bar (`|`) between the first two arguments.

A helper you name `sdt_cdf` therefore cannot be called as `sdt_cdf(x, mu)`; it has to be `sdt_cdf(x | mu)`. If the function is not really a distribution function, give it a different name — `sdt_cumprob` — and the problem disappears. `bmm`'s own distribution functions take the other route and use the bar: `swald_lpdf(rt | drift, bound, ndt, s)` and `swald_lccdf(rt | -drift, bound_lower, ndt, s)` in `inst/stan_chunks/cswald_*.stan`.

An nlf() right-hand side is pasted together as text, so operator precedence is yours to manage. The IMM builds its `theta` expressions with glue and parenthesizes every sub-expression, including the constant: `"{lure_idx} * (-expS*{nt_distances} + c) + (1 - {lure_idx}) * (-100)"` (in `configure_model.imm_full()`). When you interpolate an expression into a larger one, wrap it.

6.8 Documentation and the checks that enforce it

`NAMESPACE` and everything in `man/` are **generated**. Run `devtools::document()`; never hand-edit them.

Exported functions need `@returns` and `@examples`. Two further requirements are enforced by tests rather than by review:

@keywords. The package website is built by pkgdown, which needs every public help topic to appear somewhere in its reference index, and `_pkgdown.yml` is where that index is declared. `tests/testthat/test-pkgdown.R` reads every `man/*.Rd` file, drops the ones with `\keyword{internal}`, and fails if any remaining topic is not covered by the reference section of `_pkgdown.yml`. Coverage can be by name or through a `has_keyword()` selector, and `_pkgdown.yml` has `has_keyword("bmmmodel")`. So a new model's user-facing alias needs `#' @keywords bmmmodel`, which every model alias carries and which `use_model_template()` does **not** generate. Add it yourself. An exported function that is not a model gets `@keywords internal` or a place in `_pkgdown.yml`.

A NEWS entry for every user-visible change, under the right heading in `NEWS.md`, with the issue or PR number in parentheses. Read the most recent section for the expected level of detail: entries say what changed, what the old behaviour was, and why it mattered, not just “fixed a bug”.

6.9 Tests

Test files mirror source file names — `R/model_ddm.R` is tested by `tests/testthat/test-model_ddm.R`. This is a convention rather than a rule: some test files cover a theme (`test-default-priors.R`, `test-pkgdown.R`) rather than a file. A new model gets a matching test file.

Use brms' mock backend for pipeline tests (Chapter 5 explains what it is and the two ways to get the call wrong) and `testthat` edition 3 (`Config/testthat/edition: 3` in `DESCRIPTION`). Set `withr::local_options("bmm.silent" = 2)` so `message2()` output does not clutter the run.

6.9.1 Parameter recovery, and the circularity trap

A new model needs a parameter recovery study. It is not part of the package sources, so it is not run by R CMD check and does not need to be fast. Report it in the pull request: the script, what generated the data, and the values you recovered. Where you keep the script is up to you — bmm's `.gitignore` excludes `local/` for exactly this kind of scratch work (Chapter 1) — but a study the reviewer cannot see does not count as one.

Do not simulate and fit with the same distribution function. If you generate data with your own `rmymodel()` and then fit a likelihood built from your own `dmymodel()`, the study will report clean recovery whether or not either function is correct — any error shared by both cancels. Use an independent generator: `rtdists::rdiffusion()` for diffusion-type models (`rtdists` is already an Imports), a published reference implementation, or a direct simulation of the process the model describes. A recovery study that cannot fail is not evidence.

Recover both the hyper-parameters (means and standard deviations) and the subject-level parameters, and report how much data was needed, because that is the question a user will ask.